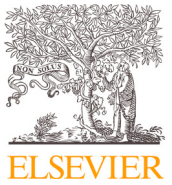
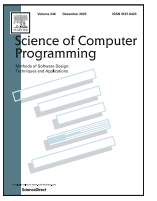
Science of Computer Programming 253 (2026) 103473



Contents lists available at ScienceDirect

Science of Computer Programming

journal homepage: www.elsevier.com/locate/scico





A survey on web testing: On the rise of AI and applications in industry

Iva Kertusha [a,*], Gebremariam Assres [a], Onur Duman [b], Andrea Arcuri [a]

[a] Kristiania University of Applied Sciences, Kirkegata 24-26, Oslo, 0153, Norway
[b] Glasgow Caledonian University, Cowcaddens Road, Glasgow, G4 0BA, Scotland, UK

ARTICLE INFO

Keywords:
Web testing
GUI testing
Frontend
Survey
SBST
AI
Fuzzing

ABSTRACT

Web application testing is an essential practice to ensure the reliability, security, and performance of web systems in an increasingly digital world. This paper presents a systematic literature survey focusing on web testing methodologies, tools, and trends from 2014 to 2025. By analyzing 258 research papers, the survey identifies key trends, demographics, contributions, tools, challenges, and innovations in this domain. In addition, the survey analyzes the experimental setups adopted by the studies, including the number of participants involved and the outcomes of the experiments. Our results show that web testing research has been highly active, with ICST as the leading venue. Most studies focus on novel techniques, emphasizing automation in black-box testing. Selenium is the most widely used tool, while industrial adoption and human studies remain comparatively limited. The findings provide a detailed overview of trends, advancements, and challenges in web testing research, the evolution of automated testing methods, the role of artificial intelligence in test case generation, and gaps in current research. Special attention was given to the level of collaboration and engagement with the industry. A positive trend in using industrial systems is observed, though many tools lack open-source availability.

1. Introduction

The World Wide Web, commonly known as the Web, is the largest internet-based application and has experienced remarkable growth since its public debut in 1993 [1]. In an annual report published by Verisign, a global provider of critical internet infrastructure and domain name registry services, it is estimated that there are 354 million existing domains in the first quarter of 2023.[1] In 2024, Statista, a global data and business intelligence platform, reports 5.44 billion internet users worldwide.[2] SimilarWeb, a company specialized on web analytics,[3] reports the top three most visited industries in 2024 are Search Engines, News & Media, and TV Movies & Streaming [2]. This highlights the significance of the web frontend, as it directly influences user engagement and interaction across these highly visited industries.

The web frontend plays a crucial role in shaping user experiences, serving as the interface between users and the underlying functionality of a web application [2,3]. A well-designed frontend enhances usability, accessibility, and engagement by providing

* Corresponding author.
E-mail address: iva.kertusha@kristiania.no (I. Kertusha).

[1] https://blog.verisign.com/domain-names/verisign-q1-2023-the-domain-name-industry-brief/
[2] https://www.statista.com/topics/1145/internet-usage-worldwide/
[3] https://www.similarweb.com/

https://doi.org/10.1016/j.scico.2026.103473
Received 29 August 2025; Received in revised form 9 March 2026; Accepted 10 March 2026
Available online 24 March 2026
0167-6423/© 2026 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

intuitive navigation and visually appealing layouts. As the first point of interaction, the frontend significantly impacts user perception and satisfaction of the web application.

Web frontend testing is vital to ensure that web applications provide a seamless and reliable user experience [4,5]. By validating the functionality, usability, and responsiveness of the user interface, frontend testing helps identify and resolve potential issues before an official release or update. This not only enhances user satisfaction but also upholds the application's reputation by preventing bugs, inconsistencies, and performance bottlenecks.

Web frontend testing presents several challenges due to the dynamic and diverse nature of modern web applications. The complexity of interactive elements, asynchronous behaviors, and responsiveness pose significant challenges in testing web frontend applications. Maintaining up-to-date test environments and addressing the rapid evolution of web technologies further complicates the testing process.

As web applications become more complex and dynamic, testing approaches must adapt to ensure reliability, usability, and performance. Web development has undergone significant transformations in recent years, introducing new technologies, frameworks, and practices that need a corresponding evolution in testing methodologies.

In this survey, we aim to identify key trends, advancements, challenges of the web testing domain over the past decade, and conduct a more in-depth analysis of these aspects. Although several surveys have been published since 2006 (discussed in detail in Section 3), none offers the depth of this study in terms of the number of papers analyzed and the diversity of aspects considered.

To achieve a comprehensive understanding of this field, we have analyzed 258 papers covering the period 2014-2025. We formulated 13 research questions designed to explore and assess the main developments and emerging trends in web testing during this period.

Because our study examines a substantially larger body of literature than previous surveys, our findings may differ even when addressing similar topics. For example, ICST, identified in our study as the most prominent venue for research work in software testing of web applications, was entirely absent from some earlier surveys. Several of the research questions we pose have not been addressed in prior work. Additionally, because web testing is a highly practical field with strong industrial relevance, we also included a research question focused on the level of industry-academia collaboration observed in the literature.

The article is organized as follows. Section 2 introduces important background information needed to better understand the rest of this article. Section 3 discusses related work. Section 4 describes the paper selection process. In Section 5 we answer and discuss our research questions. Our reflections and insight from answering these research questions are presented in Section 6. Threats to validity are discussed in Section 7. Section 8 provides the conclusion of the article.

2. Background

2.1. Web applications

Since its introduction in 1991 [6], the web has undergone significant evolution, marked by continuous advancements. The web is quite complex, comprising of, but not limited to, frontend technologies, backend technologies, database technologies, protocols, etc. Initially, all Web documents were simply HTML pages [6]. The introduction of Cascading Style Sheets (CSS) enhanced the presentation and layout of web pages, while JavaScript enabled dynamic and interactive content [6]. The development of server-side technologies and databases facilitated the creation of complex web applications, expanding the web's functionality beyond static content delivery. In recent years, trends such as Progressive Web Applications (PWAs), responsive design, and the integration of artificial intelligence have shaped web development practices. Web 4.0 [7] is the current phase of internet, often referred to as the "Intelligent Web". It represents the next phase of internet evolution, integrating artificial intelligence, machine learning, and advanced automation to create highly personalized, context-aware, and interconnected digital experiences [7].

2.2. Javascript and asynchronousness

JavaScript is a fundamental technology of modern web development, playing a crucial role in creating dynamic and interactive web applications. Initially developed in 1995 by Netscape, JavaScript was designed to enhance static web pages with simple scripting capabilities [8]. According to Statista, JavaScript is the most popular language of 2024.[4] Over time, JavaScript has evolved into a powerful language that operates both on the client and server sides. Popular and powerful JavaScript frameworks include React, Angular, and Vue.js in frontend development. The rise of Node.js has extended JavaScript beyond the browser, enabling server-side development. JavaScript is a cornerstone of web development, continuously evolving with new tools, libraries, and performance optimizations to support complex web applications and real-time interactivity.

JavaScript possesses several features that make it appealing to developers, including dynamic typing, support for event-driven programming, asynchronous event handling, first-class functions, and its weakly typed nature. These very same characteristics make JavaScript challenging to test. Its dynamic nature introduces complexities such as unpredictable type conversions, runtime errors, and difficulties in handling asynchronous operations.

[4] https://www.statista.com/statistics/793628/worldwide-developer-survey-most-used-languages/

Table 1
Existing surveys about web testing. We report the year of publication, and how many papers were analyzed. When this latter information is not specified, we report in parentheses () the total number of references in such articles. We also report the time-period of the analyzed articles, as well as specifying if the articles focus only on some specific aspects of web testing.

Year	Reference	Nr. Papers	Period	Focus
2006	[18]	(27)	All previous	
2013	[4]	79	2000–2011	
2014	[19]	(51)	Previous decade	
2014	[5]	95	2000–2013	
2014	[20]	(82)	Previous 2 decades	
2014	[21]	(237)	All previous	Extraction GUI Models
2016	[22]	(118)	Previous 2 decades	Testing&Analysis
2016	[23]	(24)	All previous	
2016	[24]	45	2006–2015	Model-Based Testing
2018	[25]	79	1988–2017	Regression Test Case Generation
2019	[26]	27	2006–2016	Quality
2021	[27]	142	2020	Grey Literature
2021	[28]	20	2009–2021	Selenium Framework
2021	[29]	12	Previous decade	Black-Box Fuzzing
2022	[30]	98	2008–2021	
2023	[31]	26	2012–2021	
2023	[32]	66	2000–2022	Testing Web Page Layout
2024	[33]	(199) 314	2014–2023	
2025	[34]	72	2013–2023	

2.3. Web testing

Testing is an important phase in the software development lifecycle, as it ensures that applications adhere to quality standards, fulfill functional requirements, and meet user expectations. Testing a web application amounts to almost 50% of the development cost [9,10]. Given the complexity of web applications now-a-days, the need for highly efficient and effective testing tools has increased.

Manual testing involves executing test cases manually without the use of automated tools to identify defects in a software application. Testers simulate user interactions to evaluate functionality, usability, and overall system behavior, making it essential for exploratory, usability, and ad-hoc testing scenarios. However manual testing is prone to human errors, limited exploration capabilities and huge costs. Given the rapid development cycles of web applications, it is essential to automate the testing process as much as possible to ensure efficiency and maintain quality. One of the earliest automated tools for web testing is Selenium,[5] which enables controlling a browser programmatically. Now Selenium has evolved into a suite of tools, including Selenium WebDriver, Selenium Grid and Selenium IDE. Being open-source, supporting multiple programming languages and its integration with various frameworks make Selenium a popular choice for developers and testers. Other tools include Puppeteer,[6] Playwright[7] and Cypress.[8] It is important to emphasize that these libraries and tools primarily serve to automate the execution of test cases by enabling controlling a browser programmatically, while the creation and design of the test cases remain a manual process carried out by testers.

This led to the need of automated test case generation. Within the academic literature there are a few tools on automation of test case generation, with some of the earliest tools being ATUSA [11], AutoBlackTest [12] and GUITAR [13]. Other popular tools include Crawljax [14], Testar [15], QExplore [16], and WebExplor [17].

There are two main principles based on which most of these tools are built on: *white-box* and *black-box*. White-box testing involves examining the internal structure, code, and logic of a system to ensure correctness. It typically requires knowledge of the software's implementation. In contrast, black-box testing focuses on evaluating a system's functionality based on input and expected output, without considering its internal workings.

3. Related work

Testing web applications is an important problem of practical value. As such, throughout the years lot of research work has been carried out on this topic. Such work has been analyzed and categorized in few surveys in the last 20 years. Table 1 provides a summary of these existing surveys.

One of the earliest work is from Di Lucca and Fasolino [18], in 2006. The state-of-the-art of web testing was discussed, although it was not a formal survey or systematic literature review. In its reference list in [18], 27 articles were cited in total. Nearly a decade later, in 2014, another two non-systematic surveys analyzed the state-of-the-art in web testing research [19,20]. A mapping study was

[5] https://www.selenium.dev/
[6] https://pptr.dev/
[7] https://playwright.dev/
[8] https://www.cypress.io/

carried out in 2013 [4], analyzing 79 published articles. This was followed by a systematic literature review by a subset of the same authors in 2014 [5], analyzing 95 articles. A short (27 articles) survey appeared afterwards in 2019, covering a similar time period, i.e., 2006–2016. A survey focused on how GUI models (including from web applications) can be extracted for automated testing was presented in 2014 [21]. These early surveys [4,5,18–21] cover the periods before 2014, which is the starting point for our survey.

In 2016, a position paper [22] discussed the latest advances in web testing in the previous two decades, but including as well other kinds of software analyses. In the same year, a systematic literature review [24] analyzed 45 articles, but focusing only on "Model-Based Testing". Another review appeared in 2016, but only included up to 24 articles [23]. In the following years, other surveys focusing on some specific aspects of testing web applications are for example about "Regression Test Case Generation" [25] (79 articles), "Quality of Tests" [26] (27 articles), "Selenium Framework" [28] (20 articles), "Black-Box Fuzzing" [29] (12 articles) and "Testing Web Page Layout" [32] (66 articles). Furthermore, besides analyzing the academic literature, there has been also cases like in [27] in which the "Grey Literature" about web testing has been analyzed (142 sources).

There are three recent peer-reviewed surveys about web testing that are close to our work [30,31,34], analyzing from 26 to 98 (but this latter going back to 2008) articles, considering a similar time period to our 2014–2025 (see Table 1), although not covering the last two years (i.e., 2024–2025). By analyzing 258 articles, we can provide a more thorough view of the last decade in web testing research. Furthermore, with 13 research questions, we do analyze several important aspects of web testing that were not discussed before. For example, considering the current success stories of artificial intelligence techniques in many different aspects of science and engineering, we make sure to analyze how they have been used in web testing (as we will discuss in Section 5.12). Also, our survey doesn't focus only on testing. Other aspects like "automated repair" of code [35] and "fault localization" [36] (which were included for example in [34]) are not analyzed here in our survey, as not in scope.

A non-peer-reviewed survey [33], published on arXiv in 2024, covers similar topics compared to our survey. In [33] it is stated that 314 articles are reviewed, but only 199 are cited. So we cannot check if in our survey we missed any, or those other articles cover different aspects of web applications that we do not consider in our survey. That survey does not cover the latest 2 years, i.e., the time-period 2024–2025. Furthermore, with only six research questions [33], there are several important research questions that we study in our survey that are not covered in Li et al. [33]. These for example include an analysis of the kinds of scientific contributions (RQ4), how studies with human subjects were performed (RQ6), how novel tools were compared (RQ8), which frameworks are most used to enable GUI-frontend testing (RQ11), the role of AI-techniques (RQ12) and details of applications in industry (RQ13).

Compared to existing surveys in the literature, our survey provides the important contribution of a more in-depth analysis of the last 12 years of research in web frontend testing.

4. Research method

4.1. Research questions

To have a better understanding of the evolution and current state of web testing, we formulated the following Research Questions, which we are going to answer in this survey.

- RQ1: How many scientific articles were published each year between 2014-2025, and in which venues?
- RQ2: Who are the most active authors?
- RQ3: What are the main scientific topics of focus in these articles?
- RQ4: What kind of scientific contributions do these articles provide?
- RQ5: What kind of SUTs are used to perform experiments?
- RQ6: In human studies, what is the number of participants involved in the experiments?
- RQ7: Which tools are introduced or improved, and released as open-source?
- RQ8: In algorithm experiments, how often are proposed tools compared with other tools?
- RQ9: In case of algorithm experiments, how many faults are found?
- RQ10: Which test generation technique is used most?
- RQ11: Which existing frameworks and/or libraries are used to enable GUI testing?
- RQ12: How many studies involve the use of AI techniques?
- RQ13: How many studies involve collaborations with industry?

RQ1 and RQ2 aim to provide a general overview of the demographics of the publications. RQ3 and RQ4 focus on identifying the main topics and contributions of the papers, acknowledging that Web Testing is a broad and diverse field with several areas of focus. In cases where the paper discusses an experiment, we discuss the used SUTs in RQ5. If the experiments involve human participants, RQ6 addresses this aspect. Through RQs 7 and 8, we aim to analyze the tools presented in the studies and understand their lifespan, usage and maintenance. RQ9 attempts to check the effectiveness of the proposed tools and techniques. RQs 10 and 11 attempt to better understand the approaches on the proposed methods and techniques. RQ10 examines the balance between white-box and black-box testing approaches, and RQ11 investigates the extent to which libraries and frameworks are used to facilitate GUI testing. RQ12 and RQ13 address two particularly important aspects: RQ12 explores the integration of AI techniques in web testing, while RQ13 evaluates the degree of collaboration between academia and industry within the selected studies. We believe that these research questions address various key aspects and provide a comprehensive understanding of the current state of the literature.

Table 2
The queries executed for each database and the resulting number of papers from each query.

Database	Query	Nr of papers
IEEE	((“All Metadata”: “web NEAR/2 test*”) OR (“All Metadata”: “web NEAR/2 fuzz*”) OR (“All Metadata”: “UI NEAR/2 test*”) OR (“All Metadata”: “UI NEAR/2 fuzz*”) OR (“All Metadata”: “GUI NEAR/2 test*”) OR (“All Metadata”: “GUI NEAR/2 fuzz*”) OR (("Document Title": "GUI") AND (("All Metadata": "test") OR ("All Metadata": testing)) AND ("All Metadata": "web")) OR (("Document Title": test* OR "Document Title": fuzz*) AND ("Document Title": "web app*"))) AND NOT(("All Metadata": fuzzy)) AND (NOT ("Document Title": mobile OR "Document Title": android OR "Document Title": iOS OR "Document Title": desktop))	phase 1 - 160 phase 2 - 37
ACM	Title: (test*) AND (NOT (Title:(mobile) OR Title: (android))) AND ((Abstract:(GUI) OR Abstract:(front-end) OR Abstract:(frontend) OR Abstract:(front end)) AND (Abstract:(test*) OR Abstract:(fuzz*)) AND (Abstract: (Web)))	phase 1 - 58 phase 2 - 5
ScienceDirect	Title, abstract, keywords: ("test" OR "testing" OR "fuzz" OR "fuzzing") AND ("user interface" OR "UI" OR "GUI") AND ("web application" OR "web")	phase 1 - 57 phase 2 - 32
Wiley	"test* OR fuzz*" in Abstract and "GUI OR frontend OR front-end OR front end OR user interface OR UI" in Abstract and "test*" in Title	phase 1 - 13 phase 2 - 0
Web of Science	(((TI = ("test*" OR "testing*" OR "fuzz*" OR "fuzzing*")) AND AB = ("test*" OR "testing*" OR "fuzz*" OR "fuzzing*")) AND AB = ("web")) AND AB = ("GUI" OR "graphical user interface" OR "UI" OR "user interface" OR "front end" OR "front-end" OR "frontend")	phase 1 - 61 phase 2 - 6
SpringerLink	("test" OR "testing" OR "fuzz" OR "fuzzing") AND ("web application") AND ("GUI" OR "graphical user interface" OR "UI" OR "user interface" OR "front end" OR "front-end" OR "frontend")	phase 1 - 236 phase 2 - 161
MIT	Title containing these search terms: testing OR test OR fuzz OR fuzzing AND Abstract/description containing these search terms: "test" OR "testing" OR "fuzz" OR "fuzzing" AND Abstract/description containing these search terms: "GUI" OR "front end" OR "front-end" OR "UI" OR "user interface" AND Abstract/description containing these search terms: "web application"	phase 1 - 40 phase 2 - 20
Total		phase 1 - 625 phase 2 - 261

4.2. Databases and search queries

To identify all relevant papers, we conducted a comprehensive search across seven major databases: IEEE, ACM, MIT Libraries, Web of Science (WoS), Wiley, ScienceDirect, and SpringerLink. These databases are widely used in the research community and contain peer-reviewed articles from reputable conferences and journals. They have been used in previous surveys in software engineering research, including [32–34].

Initially, we identified key terms pertinent to our research objectives to ensure the retrieval of relevant papers from each database. Using these keywords, we constructed search queries. Since each database employs a unique search interface and format, we adjusted the queries accordingly. Furthermore, we applied additional filters specific to each database to refine the search results. For example, in MIT Libraries, we utilized filters for publication date, language, and material type (e.g., books, journals, articles).

Table 2 shows the queries we run in every database.

4.3. Paper selection criteria

Our initial search, conducted in 17–24/04/2024 resulted in 625 papers. This large number can be attributed to the fact that our main keywords are quite common and appear even in papers that might not be related to web testing. We decided to eliminate the papers that did not fit the criteria below.

- Paper is in English.
- Paper is published between 2014–2024.
- Paper is related to web testing.
- Paper is not a thesis or a patent.
- Paper is not a survey or a systematic literature review

The first two authors went over all the papers independently, decided on the list of papers to be included, and then compared their respective lists. In case of any discrepency, the issue was solved by the fourth author. After checking all the papers retrieved initally from the databases, we ended up with 145 papers. These papers were reviewed again from the authors, and another 21 papers were excluded.

The second phase in paper collection involves snowballing. We followed the guidelines described in [37]. We conducted forward and backward snowballing, and we ended up adding 90 new papers. Our final list amounts to 214 papers. Based on this selection of articles, a first draft of this survey was published on arXiv [38].

To get a better picture of the current state-of-the-art in web testing, in particular to take into account the latest development in AI-based testing, we went through a third phase during 12-15/05/2025, where we added and analyzed papers published in 2024 and

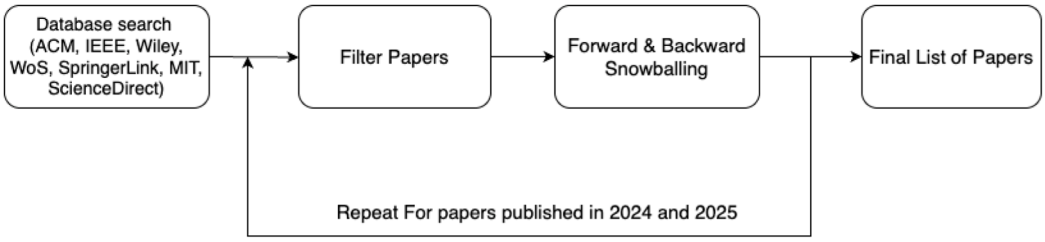


Fig. 1. Paper selection process.

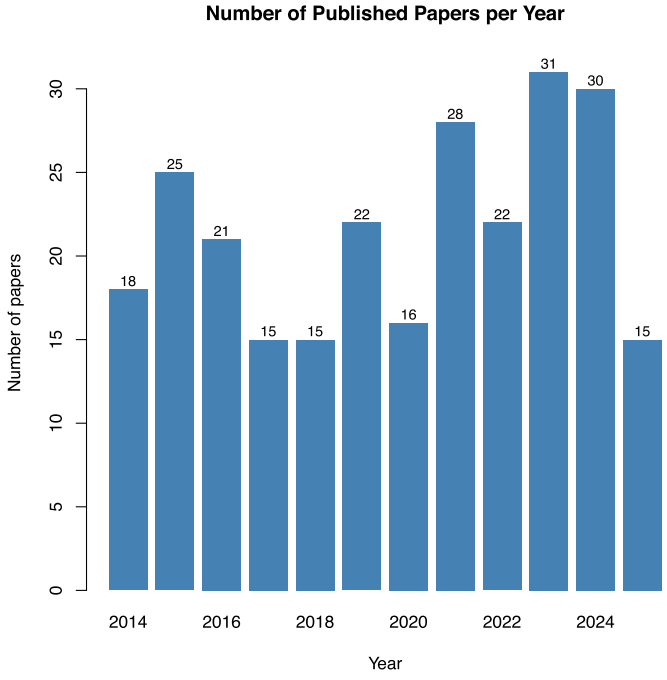


Fig. 2. Number of published papers per year.

2025. In this phase, the search yielded an additional 261 papers. We executed again the steps described in phases one and two, this time setting the publication years 2024 and 2025. After the third phase, the total number of papers included in this study is 258. This process is illustrated in Fig. 1.

4.4. Data extraction

The study comprises an extensive collection of papers. To manage and analyze the data, a structured Google spreadsheet was created. This document contains the list of selected papers alongside the extracted data, which is organized to address the research questions outlined in Section 4.1. The document is available online on Zenodo.[9] Given the extensive collection of papers analyzed in this study, the dataset was divided between the first two authors for initial review and data extraction. Upon completion of the spreadsheet containing all extracted data, the results were collaboratively reviewed and thoroughly cross-verified by the remaining two authors to ensure accuracy and consistency.

5. Empirical study

5.1. RQ1 - How many scientific articles were published each year between 2014-2025, and in which venues?

Fig. 2 illustrates the publication rate of papers on web testing from January 2014 to May 2025. The publication rate has remained relatively stable, fluctuating between 15 and 31 papers per year, with the peak reached in 2023. In 2025 up to May, 15 papers have already been published, indicating a promising trend that the rate of publication is likely to continue without decline.

[9] https://doi.org/10.5281/zenodo.18778583

Table 3
Top 15 most mentioned authors.

Author	Count
Filippo Ricca	28
Maurizio Leotta	24
Andrea Stocco	16
Paolo Tonella	15
Ali Mesbah	13
Rahulkrishna Yandrapally	7
Matteo Biagiola	6
Tanja EJ Vos	6
Boni García	5
Dario Olianas	5
Diego Clerissi	5
Emil Alégroth	5
Leonardo Mariani	5
Riccardo Coppola	5
Tommaso Fulcini	5

Table A.10 shows all the venues in which the different articles have been published. These venues are sorted based on the number of published articles. For each venue, it is indicated if it is a journal, a conference or a workshop. Due to the large number of different venues (more than 100), we only report the acronym of the venues in which at least two articles have been published. All others are grouped together into the "Other conferences" and "Other journals" categories.

From this list, the most common venue is ICST: the IEEE International Conference on Software Testing, Verification and Validation. Furthermore, the 4th most common venue is ICSTW, which is the set of workshops run at ICST. Arguably, ICST is among the most prestigious conferences in software testing. The 2nd and 3rd common venues are FSE (ACM International Conference on the Foundations of Software Engineering, previously known as ESEC/FSE) and ICSE (IEEE/ACM International Conference on Software Engineering). These are arguably the most important conferences in software engineering, in general (and not specific to testing).

Regarding journals, the most commons are JSS (Journal of Systems and Software), STVR (Software Testing, Verification and Reliability), SQJ (Software Quality Journal), TSE (IEEE Transactions on Software Engineering), EMSE (Empirical Software Engineering), IST (Information and Software Technology) and TOSEM (ACM Transactions on Software Engineering and Methodology). These are arguably the most important journals focused on software engineering and software testing.

This data shows a clear importance of web testing and the major efforts of the research community in addressing its research challenges.

Compared to other surveys covering the same period like [34], we see similar trends when it comes to journals. However, our data is significantly different when it comes to conferences. For example, in [34] the most common venue ICST (and its workshops ICSTW) is not even mentioned.

RQ1: The field of research on Web Testing has been significantly active in the last decade, with at least 15 articles published per year, and no significant signs of any slow down. Articles are published across a wide range of venues, including both conferences and journals. ICST is the most common venue where to find the latest advances in Web Testing research, followed by FSE and ICSE. The top journals in software engineering all share a similar amount of articles on Web Testing.

5.2. RQ2 - Who are the most active authors?

Table 3 presents the most active authors in the field, along with the number of papers by each author included in this study.

Filippo Ricca emerges as the most prolific author, with 28 papers. Ricca has frequently collaborated with Maurizio Leotta, Andrea Stocco, Paolo Tonella, Matteo Biagiola, and Riccardo Coppola, resulting in several joint publications such as [39–41] etc.

Ali Mesbah is another prominent contributor to the field, widely recognized as one of the main authors of the Crawljax tool. He has collaborated with RahulKrishna Yandrapally on multiple studies like [42,43].

In Stocco et al. [43] and Stocco et al. [44], we see another collaboration between Andrea Stocco, Rahul Krishna Yandrapally, and Ali Mesbah. In these articles the authors propose a test repair technique implemented in the tool *Vista*.

Tanja Vos is a well-established researcher, primarily known for her work on automated testing techniques and for her contributions to the TESTAR framework.

RQ2: We were able to identify the 15 most active researchers in the field. Their contribution varies from different test generation techniques to test maintenace, gamification techniques etc.

5.3. RQ3 - What are the main scientific topics of focus in these articles?

Testing web frontend applications present many challenges. As such, several different articles in the literature addressed different problems in this domain. The following is qualitative categorization of these addressed topics. As for all other qualitative categorizations in these articles, it was first proposed by one of us authors after reading all the reviewed articles, and then refined via discussions with all other authors. Each article's category was reviewed by at least two of us authors. An article can focus on one or more topics/contributions. Those categories are as follows.

Test Generation: articles involving the automated generation of new test cases.

Test Maintenance: articles that focus on the maintainability of existing test cases. Web applications evolve, and ideally existing test cases should fail only if revealing faults, and not for other reasons.

Testing Framework: articles related to how test cases can be (manually) written and/or executed.

Model Inference: articles describing how models can be derived from web frontend GUIs. These models can then be used for different purposes, typically for test generation.

Oracle: articles presenting or evaluating new techniques to automatically detect faults in web applications.

Coverage Criteria: articles presenting new or evaluating existing coverage criteria for web testing.

Test Prioritization: and selection, related to regression testing for web applications.

Taxonomy: articles presenting new taxonomies related to web testing.

Dataset: articles presenting and discussing curated datasets to enable and simplify future research studies.

Table A.11 shows, for each observed combination of contribution, the number articles in that group, with references. Table 4 is based on the same data, but only showing the number of articles for each contribution in isolation. From this data we can see that, by far, the most common topic is automated test generation. Given a web application, many techniques have been proposed to automatically generate test cases, using for example libraries/frameworks such as Selenium to control a browser to interact programmatically with the web application. Different strategies can be designed to best explore the graphical user interface (GUI), where transitions from page to page can be represented with models. Different tools/techniques build models of the GUI to enable a more efficient test generation. However, inferred models can be used also for other activities.

The investigation of frameworks/libraries for writing and executing tests is common as well. Related to generating tests, there has been novel work on how to automatically detect faults in web applications (i.e., oracle problem), as well as to define coverage criteria for web test cases. Test priorization (and selection), as well as the definition of taxonomies, have received comparatively only little attention. We found only two articles dealing with the collection and specification of a dataset for experimentation in web testing [92] and [93].

An important topic of research is test maintenance. Once a test case is manually written, or automatically generated, how will it be used in future versions of the evolving tested web application? This topic category can be further clustered in three main groups (*locator*, *page-object* and *flaky*), as shown in Table 5. Most of the work has been dealing with HTML *locators*. To interact with a

Table 4
Categories and total papers per category.

Category	Nr.Papers
Test Generation	124
Test Maintenance	52
Testing Framework	48
Model Inference	35
Oracle	21
Coverage Criteria	14
Test Prioritization	7
Taxonomy	4
Dataset	2

Table 5
Maintenance.

Maintenance	Nr.Papers	List.papers
locator	29	[43–52,52–70]
other	8	[42,71–77]
flaky	8	[78–85]
page-object	7	[41,86–91]

browser, a test case need to specify which buttons to click, and which forms to fill in, for example. This is done by locating on the HTML page the different widgets that a user can interact with. Each element in the HTML page needs to be uniquely and un-ambiguously identified and located. This is typically done with XPath Locators[10] or CSS Selectors.[11] A problem, though, is that the frontend of a web application can change significantly (e.g., to improve its design), without changing its functionalities. As such, HTML element locators can easily become outdated, leading the test cases using them to fail, although no semantic change was applied in the web application. This is a major problem in regression testing. A failed test case should identify a regression fault, and not just a design change in the frontend. Manually investigating these false positives can be quite time consuming and expensive. Therefore, lot of research has been carried out on how to design (and automatically create) more "robust" locators. Some techniques to handle this issue are for example involving multiple locators with a weighted similarity score [45], or by leveraging historical information [46].

The use of locators is essential when writing test cases for web frontend GUIs. However, as these locators can be quite complex, they can make the test case harder to read and understand. Furthermore, when a locator needs to be updated due to a design change in the frontend (e.g., a button has been moved into a different position in the page), that update has to be done in every test case using it. To improve readability and reduce maintenance costs, the *Page Object*[12] design pattern can be used. Research work has been carried out to design automated techniques to generate such page-objects [41].

Flakiness is a major problem in software testing [94]. Even when the tested application is not changed, re-running a test case might lead to a different result. This is a major issue for maintenance, as regression tests might start failing even if no new fault was introduced in the tested application. There can be many sources of flakiness, especially when dealing with non-deterministic events. Web applications have their own specific sources of flakiness. For example, after an event is triggered in a web application, a non-deterministic amount of time might pass before the HTML of page is updated before the next operation in the test case can be executed. Somehow, a test case has to "sleep/wait" some (milli)seconds between two distinct actions. A timeout that works fine might all of a sudden fail if for any reason a browser takes longer to update a page. Novel techniques have been designed for example to automatically replace "sleep" operations with more robust awaiting mechanisms [79].

RQ3: From a qualitative categorization of the selected articles, automated test generation is the most common problem studied in the research literature for web testing. Several other topics have been studied as well, including test maintenance, automated oracles and coverage criteria. Test maintenance is an important topic that has been receiving quite a lot of attention from the research community, with 3 main subcategories being locator, flakiness and page-objects.

5.4. RQ4 - *What kind of scientific contributions do these articles provide?*

In this research question, we look at what kind of contributions the different surveyed articles provide. This is a *qualitative* categorization, aimed at clustering the different articles based on their contribution. Each category was first defined by one author, and then refined throughout different iterations with all the other authors.

The categories we used to classify such articles are as following. Note that an article can provide one or more main contribution categories. As the name of the category alone might lead to ambiguities in interpretation, we also discuss each of them.

New Tool or Technique (NToT): this represents articles that present a novel tool or technique aimed at testing web applications. This category does not include "tool papers" (i.e., articles discussing an already published work, by presenting technical details), as those are in a different category.

Algorithm Experiment (AE): the articles present empirical studies with tools or techniques. These studies are run by the authors of the article, not involving external human subjects (e.g., students and practitioners).

Human Experiment (HE): these articles provide empirical studies with human subjects, which are not the authors of these articles. Usually, those involve studying the human aspects of employing testing techniques and tools in practice.

Position Paper (PP): these articles provide reflections, introspections and new ideas, without providing a novel tool or technique. In this group we consider as well "doctoral symposium" and "tool demo" articles.

Experience Report (ER): description of the use of an existing tool/technique in industry, typically without a formal, in-detail empirical study. This would be different from an Algorithm Experiment or Human Experiment done with industrial systems, although the line between them could be considered thin and up to interpretation. This group includes "case study" and "action research" studies, when explicitly mentioned in the text.

Analysis of Open-Source Projects (OSA): in these articles, authors do not present novel techniques, but rather studies properties of existing open-source projects (e.g., how are E2E test written? which libraries are used?).

[10] https://www.w3.org/TR/xpath/
[11] https://www.w3.org/TR/selectors-4/
[12] https://martinfowler.com/bliki/PageObject.html

Table 6
Contributions and total papers per contribution.

Contribution	Nr.Papers
New Tool or Technique	200
Algorithm Experiment	180
Human Experiment	30
Position Paper	20
Experience Report	9
Analysis of Open-Source Projects	7
Survey and Interviews	4

Survey and Interviews (survey): surveys of human subjects (e.g., practitioners in industry working with web frontend testing), including as well interviews.

Table A.12 shows each combination of category, with number articles and references. Table 6 shows each category in isolation.

From this data, we can see that, by far, the most common type of contribution is proposing a new tool or technique for web testing. In the majority of cases, when in an article a new technique is proposed, it is also empirically validated by the authors. However, there are still several cases in which no empirical study is carried out when proposing a new technique. In these cases, the work might mention an empirical study, but little to no information is provided.

Comparatively, there is only few articles involving experiments with human subjects. Experience reports in industry are even less common. Analyses of existing practices among open-source projects are only a few. Surveys and interviews of practitioners are very rare.

From this data we can see a clear trend. It looks like the research work on web testing is still in an initial academic state, where novel techniques are presented to address several different problems in web testing. However, these techniques are mainly evaluated "in the lab", directly by the authors of these novel techniques. How practitioners would use such novel techniques and tools, and all the related "human-aspects" challenges involved, are not commonly studied in the scientific literature. These are important research questions that, comparatively, have not received enough attention in the last decade.

RQ4: Most of the research work on Web Testing still focus on designing novel techniques and tools, which are then evaluated "in the lab". Their application in industry, and the study of their use among human subjects, have received significantly less attention.

5.5. RQ5- What kind of SUTs are used to perform experiments?

When designed a novel technique for web testing, or when comparing existing techniques, the choice of which web applications to use for the experiments is of paramount importance. Without sound empirical analyses, it is not possible to properly evaluate the effectiveness (or lack thereof) of a novel technique. Therefore, it is important to study how empirical analyses have been carried out in the literature.

In our survey, we have categorized four different groups of application types, as explained next. An empirical study could comprise several applications of different categories.

Artificial: small, artificial applications developed directly by the authors of these studies.

Open-Source: web applications available on open-source repositories (e.g., GitHub), which are then downloaded and started on the local machines of the article authors.

Online: web applications available on the internet, that anyone can access.

Industrial: industrial web applications, usually tested and used as case study as part of industry-academia collaborations. Note: to fit in this category, and not be labeled under "Online", a collaboration should be explicitly mentioned in the research articles. This is usually easy to determine when industrial partners are co-authors in these articles. A main difference from "Online" category is that here the engineers of these web applications are involved in the studies, and can give feedback to the academics.

Table 7 shows, for each application category, how many articles use at least one of these kind of applications. Here, for this analysis we consider only the 180articles involving "Algorithm Experiment" (recall Section 5.4).

As empirical studies might involve more than one application type, we also report the sum of the different types, as a total. For each entry, we report statistics on how many web applications were used, including the minimum (which by construction would always be at least 1), the arithmetic mean, the median and the max number.

From this data, we can see that artificial examples are not often used, and, when used, there are only a few (mean value 2.5). However, in 25 of these articles *only* artificial examples are used in their empirical studies [14,70,72,85,95–115]. This poses significant

Table 7
Types of SUTs.

Type	N	%	Min	Median	Mean	Max
Artificial	26	14%	1	1	1.8	10
Open-Source	95	52%	1	5	5.1	30
Online	56	31%	1	5	156.5	6000
Industrial	31	17%	1	1	2.1	20
Total	180	100%	0	4	52.0	6000

external validity threats for these studies. The largest use of artificial applications can be found in [116], where 10 artificial examples were used. However, those were "*J2EE web applications built by professional software engineers*" [116]. That study involved the testing of user behavior for synchronous web applications, using Petri nets. Although 10 artificial examples were used, the study also involved 8 open-source projects. As these 18 web applications were small (up to 4 554 LOCs, for a total combined of 30 186 LOCs), "*(to) address scalability, we worked with a company local to our university to apply our technique in an industrial setting*" [116]. Artificial examples can be a useful addition to an empirical study, as long as their shortcomings are addressed by using other systems as well.

The most common type of applications are open-source, where on average 5.2 applications are used. This is a relatively small number, highlighting the difficulty of collecting and enabling suitable web applications for experimentation. This, unfortunately, might have a negative impact on how likely these proposed novel techniques could fare on new applications. The largest use of open-source projects can be found in [117], where 30 open-source applications were used. This study also included 4 artificial applications, for a total of 21 web applications used in the study.

We have collected all open-source SUTs mentioned in the papers selected for this work, and summarized the information in Table A.14. In total we have identified 50 SUTs that have been used repeteadly as a benchmark in different studies. The top 10 SUTs are `addressbook`, `claroline`, `petclinic`, `mantisbt`, `mrbs`, `ppma`, `dimeshift`, `collabtive`, `splittypie` and `pagekit`. The SUT `addressbook` appears in 34 papers, making it the most used SUT. Collecting more data on the SUTs comes with a few challenges. It should be noted that links to the SUTs' source code or releases are not provided in all studies. In addition, even when a link is given, it might be no longer available, or the SUT may differ from the version used in the study (a release is not always available).

A collection of open-source projects is examined in SERGIO et al. [118]. Although the primary focus of the study is a large-scale empirical analysis of the adoption and maintenance of GUI testing, it represents a valuable step toward establishing a curated set of open-source SUTs. It is also worth noting that the dataset employed in this study builds upon the E2EGit dataset [93].

When it comes to web applications available online on internet, a third of studies involves them. The median of number of online applications used as SUTs is 5 in these studies. However, the average is very high (175), due to few outliers. In particular, there are 3 articles that involve 1000 or more SUTs [119–121]. On the one hand, not having to download and install applications locally (including setting up all related services, such as for example databases), and just rely on knowledge of the URL of the homepage of a web application, enable the possibility of larger studies. On the other hand, using online SUTs has its own shortcomings. First, web applications can change at any time, and so it makes harder to enable the *replication* of these studies. Second, there might ethical concerns when generating test cases toward an online application, as it can (possibly involuntarily) result in a *denial of service* attack if too many test cases are generated. The largest use of online applications can be found in [120], where 6000 applications were used. These large number of web sites was needed for training machine learning models aimed at "*allowing the test scripts to automatically adapt to these eventual changes of the web pages*" [120], i.e., for test maintainability.

As a sign of maturity of the field, we can see that 31 studies involve industrial applications. On average, only 1.5 industrial applications is used in these studies, with median value being 1. This means that the majority of these studies only uses 1 industrial application, with only a few having more than 1. Access to an industrial application not only requires a technique or tool that can actually scale and be applicable on real-world applications, but also considerable effort in establishing industry-academia collaborations [122]. The largest use of industrial applications can be found in [69], where 20 industrial web applications were used. In [69], the Cytestion tool was introduced: "*an approach that systematically explores a web-based GUI and dynamically builds a test suite for detecting faults that cause visible failures*". In the empirical study, 4 open-source projects were used, as well as 20 industrial ones. Those "*projects were React-based applications developed by different teams of a partner software developing company and referred to a web-integrated business platform, each controlling specific tasks aiming at reducing fiscal risks and costs*". Those were large systems, from 80 to 246 thousand LOCs. Evaluations in industry are essential for sound empirical studies, to verify that academic techniques can actually be usable in practice. However, to ease future comparisons, and to reduce validity threats regarding considering applications from only a single enterprise, empirical studies should involve open-source projects as well. The work in [69] is a wonderful example of what can be aimed at for what to use in an empirical study in web testing.

RQ5: It is important for an algorithm and/or a tool to be scalable and of real-world relevance. Hence we took in consideration how the experiments were conducted and what kinds of SUTs were used for the experiments : artifical, open-source, industrial or online. In algorithm/tool comparisons, on average only 5.2 open-source projects are used. Although this could be considered as a small number, we can see a promising, positive trend in using industrial systems for the empirical studies, done in 31 articles.

5.6. RQ6 - In human studies, what is the number of participants involved in the experiments?

Out of 258 papers, only 34 papers conducted experiments involving human subjects such as academics (students or faculty), or industry practitioners. In the remaining papers, any experiments conducted were carried out by the authors themselves. There are 419 academics and 503 industry practitioners that have participated in these studies. However, it is important to note that papers [123] and [124] do not specify the number of participants in their experiments. Table A.13 presents the papers that include experiments conducted by human subjects. These papers explore a variety of categories, which are detailed in Section 5.3.

While the majority of papers focus on experiments with either academics or industry practitioners, [125–129] and [130] have included both.

In the remainder of this section, we review each article that includes a study involving human participants and provide a brief summary of the study's objectives as well as the specific role participants played in the experiment.

An empirical study presented in Cassino et al. [127] evaluates the effectiveness of USherlock, an automated usability evaluation technique. The study involved 24 students and 4 expert evaluators, aiming to assess the adaptability of USherlock for users with varying levels of expertise, including those with medium proficiency.

Recheck, the tool proposed in Leotta et al. [131], implements differential testing and was evaluated through an experiment conducted by a professional tester. The evaluation involved a comparison with Selenium WebDriver, and the findings demonstrate that Selenium WebDriver tests equipped with Recheck's oracle mechanisms are more effective in detecting bugs.

In [132] the authors have asked 20 Master students with prior knowledge on web testing to record the time it takes to generate test scripts adopting Page Objects for web application. The students were given the freedom of choosing a web application for testing.

The focus in [123] is evaluation the effectivenes of TestComplete tool in GUI Testing. The number of participants is not specified. However it is mentioned that they are senior engineers. The participants had no previous experience with TestComplete.

In [128], the authors address the challenge of concurrent activities in web applications, with a particular focus on the synchronous request-response cycle, utilizing a novel Petri net-based model. The study involved three participants-one from academia and two from industry-tasked with designing and executing tests both with and without the support of WAPG, the tool developed by the authors. The results indicate tht the number of faults detected from the tests developed with WAPG is higher than the number of faults detected from the tests developed by the participants.

In [133] the authors focus on automatically generating test data for web application. Two testers were ask to perform this task using the approach introduced by the authors. The authors then compared the time it took each tester to perform the task and concluded that their approach significantly reduces the time it takes to generate test data.

The exploratory case study presented in [134] compares the benefits of scripted and scriptless testing approaches, utilizing Testar and Selenium as representative tools for each method. Among the six participants in the study, only two had prior experience with manual testing. The findings indicate that both approaches offer distinct advantages and are complementary in nature.

The framework proposed in [50] aims to enhance the resilience of test cases to UI changes, thereby reducing the costs associated with test maintenance. To evaluate the framework, an experiment was conducted involving three software developers who were tasked with creating a set of test cases for a simple web page. The web page was then manually modified to produce three different versions. The objective was to assess whether the same test cases could be applied across all three versions and to evaluate the impact of these changes on the test cases. The results demonstrate that the framework effectively supports the creation of resilient test cases, provided that the changes made to the application do not alter the core functionality.

In [135] five students, either undergraduates or recent graduates, participated in the experiments conducted on five SUTs. Three students had Computer Science background and only one of them had testing experience in the industry. However, the results of the experiments indicated that the tester's background does not influence the tool's effectiveness; instead, it is the tester's level of testing experience that proves to be a more important factor.

Article [136] focuses on feature extraction and involves five practioners who have some experience with programming. The participants were tasked with manually extracting features as well as utilizing the tool developed by the authors. The authors subsequently recorded the time taken by the participants in both scenarios to calculate the labor costs. It was conculded that using their tool, the time cost on feature extraction is significantly reduced.

The approach presented in [137] focuses on generating test cases from inferred models, implemented through the tool MoLeWe. This approach was evaluated in an experiment involving 18 students, each of whom designed a test suite consisting of ten test cases. These test suites were subsequently used as input for MoLeWe to generate additional test cases. The results demonstrate that the tool is cost-effective and that the test cases generated by MoLeWe significantly increased line coverage compared to manually written tests.

In [138], the authors present a tool designed to enhance state exploration and identify unique URLs. The performance of the proposed tool is compared against two alternative methods-random testing and Q-learning-based testing-as well as with three human participants. Among the participants, one had prior experience in testing, while the other two were novices. The results, based on the number of unique states explored by each method and participant, demonstrated that the tool performs comparably to an experienced human tester. Notably, the tool achieves these results with significantly greater speed.

In [126], the authors introduce a predictive model designed to assist testers in diversifying their testing efforts. To evaluate the model, two separate experiments were conducted. In the first experiment, participants were students enrolled in a software engineering course. The students were divided into 13 groups, each consisting of three members, and tasked with performing exploratory testing on the Cdiscount web application. Each group participated in two 10-minute sessions: one without the tool developed by the authors and one with it. The results demonstrated that the tool effectively supported testers in generating more diverse tests. In the

second experiment, the participants were practitioners from CIS Valley, with varying levels of knowledge in software testing. Similar to the first experiment, the participants conducted one session without the tool and another with it. A survey conducted some time after the experiment, indicated that the tool was again found to be beneficial in enhancing exploratory testing efforts

Article [47] introduces WebRR, a record-and-replay tool. While the primary objective of the experiment did not involve human interaction with the tool for conducting tests, the authors engaged a tester from their industrial collaborator to participate. The tester's role was to replicate the tests using both WebRR and Selenium IDE, and record the number of broken tests identified by each tool.

The study in [77] focuses specifically on test maintenance and test comprehension. The proposed solution is implemented as a Scout plugin and evaluated through an experiment conducted in collaboration with an industrial partner. Twelve participants with diverse backgrounds were asked to complete a survey assessing and comparing the reports generated with and without the proposed approach.

Scout tool was used also in [139], where the authors investigate the impact of gamification techniques on manual exploratory testing. The study involved 144 participants, all of whom were students enrolled in a Software Engineering course. Most participants had between one and three years of experience in web application development and software testing. Two web applications were selected as SUTs, and the participants conducted tests using the Scout tool, both with and without gamification features enabled. The findings indicate that gamification significantly improves test coverage. However, it was also observed that while test effectiveness remained unaffected, efficiency was negatively impacted by the inclusion of gamification elements.

Three additional studies, [125,140] and [141] investigate gamification by conducting similar types of experiments involving participants from academia. In both [125] and [140] the authors use Scout tool. In [125], 10 participants are recruited, two of which young software developers, whereas the other eight were master students. The participants were divided in two groups, one for each SUTs. Each group conducted the experiment with and without the gamification plugin on Scout tool for 30 minutes. In [140], the authors investigate the combination of gamification and crowdsourcing for web testing. For the experimental evaluation, six students were recruited and divided into two groups. Each group conducted testing on two websites, one using the proposed approach and the other without it, to assess its effectiveness.

In [141] the authors introduce another tool for gamification, GIPGUT. Unlike the previous studies [125] and [140] where the authors used Scout, in this article the authors build a plugin for IntelliJ IDE. The authors recruited four students to test Amazon website using the proposed tool. The experiment was then followed by a survey to evaluate the performance of the tool and provide feedback.

In [48] the authors aim to improve the testability of Web applications by using hook-based locators. 6 students, organized in three groups, participated in the experiment. They were responsible to develop a web application with several releases, using the process defined by the authors. Next, the generated test cases with hook-locators were executed and their fragility compared to the generated test cases with default Katalon locators. It was concluded that hook-based locators reduce the fragility of test cases and of the broken tests.

Article [124] focuses on Pattern-based GUI testing. The participants are students, however their exact number is not specified. The students were assigned to two teams, and each team was assigned one subject study. The students were introduced to PGBT and the PARADIGM-ME tool. Using the PARADIGM-ME tool, the teams began building models to achieve the testing objectives. During the study, they documented the time required to build and configure the models. Next, they generated test cases and executed the tests within the PARADIGM-ME environment. Finally, they analyzed the test results, recorded the identified failures, and concluded the case study.

Article [142] addresses test case prioritization. For the experiment, the authors selected two open-source web applications. 28 students, organized in seven groups were engaged in the experiment. The students were tasked with simulating a realistic software development process using the test automation tool Katalon Studio to create and modify test cases while adapting the application under test (AUT) across multiple iterations. Their responsibilities included generating automated test cases, executing tests, and analyzing the results to build a coverage graph.

The study presented in [129] focuses on A/B usability testing and involves the execution of three distinct case studies. In the first case study, two e-commerce web applications were selected as experimental objects, with a participant group of 49 students. The second case study examined a traffic ticketing web application, while the third case study investigated a shoe shopping e-commerce website. For both the second and third case studies, 16 participants were recruited for each experiment. However, the authors do not specify whether these participants were students, practitioners, or individuals from other backgrounds.

Article [40] is an opinion survey concentrated on Selenium WebDriver and its challenges. The participants in this survey were 78 industry professionals experienced in Selenium. The survey results provide valuable insights into the challenges developers and testers encounter, and the strategies they employ to address and resolve issues arising from the use of Selenium. Another survey focused on Selenium is discussed in García et al. [143]. In this survey a total of 72 industry specialists participated.

In [144], the authors conducted a survey to evaluate the perception and the willingness of practitioners to adopt their proposed approach - a tool designed to assist the development process with Selenium. A total of 148 practitioners participated in the survey, offering valuable insights into the use and perception of WebDriverManager. Another study focused on WebDriverManager is presented in [145], in the context of multi-browser test suites. This study involved 25 master's students and demonstrated that WebDriverManager significantly reduces the time required to set up or update multi-browser test suites.

The study presented in [87] conducts an empirical investigation to evaluate the costs and benefits of incorporating Page Object (PO) patterns in web testing. A total of three experiments were carried out, involving 36 participants comprising 19 master's and 17 PhD students. Each participant was tasked with designing a set of test cases both with and without the application of the PO pattern.

Following the experiments, the participants completed a questionnaire to provide feedback on their experience. The results indicate that adopting the PO pattern is particularly beneficial in scenarios involving large test suites.

In [146], the authors invited 20 industry practitioners to utilize the framework they developed, MAJD, and subsequently completed a questionnaire designed to assess the ease of use of the framework.

In [147], the authors aim to evaluate the performance of the TESTAR tool within the context of an industrial application developed by SOFTEAM, a French software company. Two SOFTEAM employees participated in the study-a senior analyst and a software developer-both with less than one year of experience in software testing. The participants received training on how to use TESTAR with their application. At the end of the experiment, they completed a questionnaire assessing the tool in terms of quality, learnability, and the likelihood of recommending it.

The authors in [148] present VETL, a tool based on a large vision-language model (LVLM) designed for automated test generation. They conducted an experiment to evaluate the tool's effectiveness in web GUI exploration. Following the experiment, five graduate students with prior experience in GUI testing were engaged. Their role was to assess whether the input text generated by the tool was contextually relevant to the corresponding webpage screenshots.

In [149], the authors propose a novel approach to test generation based on the evaluation of node importance. They formulate two research questions, and to address one of them, they involve five professionals. The participants are asked to assess and assign importance scores to individual nodes, which are then compared to the scores generated by the proposed tool.

The survey in [76] investigates developers' experiences with UI testing in CI/CD pipelines, identifying key challenges, adopted strategies, and perceived impacts on software delivery. To recruit participants, the authors conducted a search on GitHub and contacted 3338 users via Email, inviting them to complete a survey. A total of 128 responses were received, of which 94 were deemed valid for analysis. In addition, the authors conducted follow-up interviews with 18 participants to gain deeper insights.

RQ6: Since web testing tools and algorithms are typically intended for a industry use, it was important to distinguish which studies explicitly considered this practical dimension. Only a limited number of articles, specifically 34, incorporate human studies involving participants from both academia and industry. These participants engage in one or more different stages of the conducted experiments, including performing manual tasks, interacting with testing tools, and providing feedback. While most studies involved human participants directly experimenting with the tool, a few instead asked participants to provide feedback and complete a survey (e.g., [50], [55]).

5.7. RQ7 - Which tools are introduced or improved, and released as open-source?

A key objective of this survey was to identify the tools introduced over the past decade within the domain of web testing. At the same time we wanted to investigate the state of open-sourceness of the tools identified.

Out of 258 papers we analyzed in this survey, 207 papers focus on a tool, either newly introduced or improved. Table A.15 provides a summary of the tools identified in this study, presenting the tool name (if specified), their open-source status, the number of papers referencing each tool, and a list of those papers.

As discussed in Section 5.3, we categorized papers based on the main challenge they focus on. Hence the tools shown in Table A.15 serve different purposes, e.g., generating test cases, prioritizing test cases, repairing existing test cases.

A total of 70 papers classify their associated tools as unnamed. Among these, 10 tools are identified as open-source, while the remaining are not publicly available.

In 3 papers [132,133,150], the authors highlight the fact that they are either planning or on the process of implementing the tool. In [132] the authors plan to implement their approach of generating Selenium test suites, as a Selenium plugin. In Bures [150], the challenge the author is tackling is the quality of the frontend by collecting several metrics. The author mentions that the prototype is being implemented as a client-side Java application. The focus of [133] is Test Data Generation for Web Applications. The authors have defined as one of their future work goals to built a tool to automatically generate test data.

It is interesting to note that only 12 tools have more than one paper dedicated, and only 3 tools (TESTAR, Apogen and FSMWeb), with more than 2. The rest have only one paper published. Here we do not take in consideration the papers that mention or use the tool as a comparison, but only the papers that are dedicated to the tool. This means we are considering articles in which a tool was first presented, and then all of the following articles in which its authors have extended the tool with new features/techniques or used in new studies.

TESTAR is an automated test generation tool that was originally meant for desktop applications, and later was adapted for web applications too. Therefore the earlier papers on TESTAR, [151,152], evaluate the tool on both desktop and web applications. A comprehensive overview of TESTAR is provided in [15]. The tool has continuoulsy been maintained and improved, as documented in [111,151–154]. Two industrial case studies are presented in [147] and [155]. Other papers related to TESTAR, like [156] have been excluded from our dataset, as they are not focused on web applications.

APOGEN is a tool that automatically generates Page Objects from web applications. The authors published one journal article [41] and two conference papers, [88] and [89], all in the same year, 2016.

Model-based regression testing stands at the core of FSMWeb and are discussed in [108,157,158]. The tool is primarily focused on regression testing of fail-safe behavior in web application and implements a Genetic Algorithm.

In the papers where tools are marked as open-source, the authors provide a link to the tool (usually a GitHub repository), and/or the experiment results (if any). However, we do not make any claims regarding the validity of the provided links, as the contents of the repositories and/or the repository itslef, may have changed since the articles were published.

We have identified 72 open-source tools. That means only 36% of the tools are open-source. In the remaining 121 papers, the authors do not disclose the tool's implementation, but only their high-level algorithm. We deem the open-source state of a tool to be quite an important feature in the research domain. By making code and methodologies publicly accessible, researchers and developers can validate findings, share innovations, and build upon existing work without barriers.

RQ7: In most cases, when authors propose a new approach, they also develop a corresponding tool to implement it. The large majority of tools are no longer actively maintained after their initial implementation with TESTAR being a notable exception. Another characteristic we examined is the tool's open-source status. The majority of the tools are not publicly available, with only 36% being open-source.

5.8. RQ8 - In algorithm experiments, how often are proposed tools compared with other tools?

Out of the 180 papers in which the authors conduct an algorithm experiment, 54 papers include a comparison with an existing approach. Table A.16 summarizes the tools used for comparison. Notably, Crawljax is the most frequently used benchmark tool, appearing in 10 different studies. Tools, including WebExplor, FraGen, Morpheus, Comjaxtest, SubWeb, Webmole, QExplore, Dante, Awet, and V-DOM, have all been evaluated in comparison to Crawljax. The prevalence of Crawljax in comparative studies can be attributed, at least in part, to its early introduction in 2008, which has provided it with a longstanding presence in the field.The second most commonly used tool for comparison is WebExplor. Both Crawljax and WebExplor are primarily aimed at automated test generation. WATER, Vista and WebEvo follow as the next most frequently used tools, with a focus on test maintenance. Additionally, it is noteworthy that seven studies have employed manual testing as a benchmark for evaluating their tools. The diversity of categories, explained in Section 5.4, for these tools contributes to the limited number of direct comparisons among them.

RQ8: Almost a third of the papers that have included an algorithm experiment, compare against an existing approach. Different categories (e.g., test generation, test maintenance) imply that a different set of tools is appropriate for a meaningul comparison. Crawljax is the most frequently used web testing tool for comparison.

5.9. RQ9 - In case of algorithm experiments, how many faults are found?

As outlined in Section 5.4, our analysis focused on identifying algorithm experiments, human studies, and any experiences authors had in evaluating their proposed approaches or tools. The evaluation metrics varied depending on the paper's objectives and the specific technique or tool under discussion. We paid particular attention to the reported number of faults, as this provided valuable insights into the effectiveness of the proposed solutions. There are 68 articles that discuss their experiments and use number of faults as a metric for evaluation. However 26 articles do not report the number. Our findings are summarized in Table A.17. As observed in the table, the majority of studies that conduct experiments, and report the number of identified faults, primarily focus on test generation.

The variation in the number of reported faults is considerable, with [159] reporting 3 faults, and [17] reporting 3478 faults.

First we will take a look at the papers that report a large number of faults in an attempt to better understand what contributes to that number.

In [17], the authors introduced WebExplor, a tool designed for automated end-to-end web testing. They evaluated the tool on 57 diverse applications, including both real-world and open-source web applications. Conducting an experiment on such a large scale across numerous applications significantly contributed to the high number of reported faults. While the authors do mention server request errors, they do not specify whether the faults and errors they report are exclusively server-side.

VETL, introduced in [148], is a large vision-language model-driven tool designed for end-to-end web testing. It was evaluated on two types of applications: four open-source websites and ten online websites. The failures that the authors report are failure logging messages collected from the browser's console. The number of detected faults varied across applications, with the highest average reaching 1782.4 failures. However, the authors note that the majority of these errors, 85.4%, were caused by insufficient memory usage. The second highest number of failures reported for a WUT was 291.2.

TOM is a model-based test generation tool introduced in [160]. To evaluate their approach, the authors conducted experiments on a single web application, OntoWorks. Their analysis reported 935 step failures, a considerable number for a single application. However, the authors noted that some behaviors initially classified as failures were, in fact, expected system behavior. Additionally, they highlighted the presence of cascading errors, where a single failure triggered a chain of subsequent failures. While the exact number of actual failures, was not specified after these clarifications, these factors likely contributed to the high reported error count.

The study in [71] primarily focuses on test maintenance. In this case, the main focus is not the number of faults found in an SUT, rather the number of test breakages when using different versions of the same web application. The authors report a total of 1065 individual test breakages across 453 versions of eight different web applications.

In [161], the tool InwertGen was evaluated on seven web applications. The authors report their subject applications and experimental results in greater detail. Specifically for experimental results, they present line, branch, and function coverage as percentages, as well as the number of newly discovered states, malformed HTML issues, and error pages. While a total of 569 malformed HTML issues is reported, these are noted as not being severe errors in themselves. In contrast, only five error pages, caused by severe issues such as page not found errors, were identified by the tool.

In [162], the proposed framework was tested on 100 web applications, with the authors reporting 255 detected defects. Very little has been reported on what is considered a defect and how they were were collected. - to extend this one

In [163], the authors highlight that the faults they focus on are faults resulting in visible failure. *"Those are faults that can be detected in the form of failing text messages presented through the GUI (e.g., errors, exceptions), through the browser's console messages, or bad results of requests made to the application's server."* In the case of industrial projects the tool was able to detect 13 faults. In the case of open source projects the tool was able to detect 129 of the 165 injected faults.

In [110] the proposed tool, PirlTest, is evaluated through experiments on both mobile and web applications. The authors report metrics such as code coverage and the number of detected bugs. The study states that the tool identified 128 unique bugs across the evaluated mobile and web applications; however, the criteria used to define a 'bug' are not explicitly specified

While the majority of papers provide the total number of detected failures, some studies instead report a percentage or an average count. For instance, in [164,165] and [159] the authors report the average number of unique failures detected. In [166] and [149], the authors present the fault-finding capability of their implementation as an average percentage. In [167], the authors emphasize that the reported numbers specifically refer to exception messages rather than other types of failures. [168] further categorizes the detected faults in vulnerabilities and crashes.

RQ9: Through this research question, we sought to examine the reported effectiveness of the proposed tools and techniques. 68articles, most of which focus on test generation, report quantitative results in terms of detected faults or errors. However, the definition of a 'fault' varies across studies, ranging from server-side errors to any type of issue reported in the browser console. In several articles, the notion of a fault is not explicitly defined. Furthermore, the reported fault counts are influenced by factors such as the number of systems under test (SUTs) included in the evaluation.

5.10. RQ10 - Which test generation technique is used most?

Out of the 258 papers included in the review, 22 focus on *white-box* testing, 170 on *black-box* testing, and the remaining cover not relevant testing techniques for ensuring the quality of web applications.

The studies focusing on white-box testing techniques extensively cover end-to-end, regression, and unit test automation in web applications [169–171]. The following summary highlights research emphasizing these techniques.

End-to-end and regression testing methods are key aspects explored, with the study in [169] emphasizing UI tests and code coverage for transcompiled applications. Similarly, the study in [168] generates test cases from client-side behavior models to address server-side vulnerabilities in PHP applications, while [72] and [172] focus on regression testing automation for PHP applications using GUI and Selenium IDE, respectively.

The studies focusing on unit testing explored coverage metrics and efficiency in test generation for web applications written in various languages. For example, the study in [170] targets function, statement, and branch coverage for TypeScript applications, while [173] and [174] propose advanced techniques for JavaScript applications.

Similarly, data flow testing is applied to Perl applications in [103], and full-stack testing approaches, such as in [171] and [175], integrate client-server models to classify errors and generate parallel test cases for PHP applications.

Studies such as in [176] and [177] focus on achieving high coverage in JavaScript testing and assessing adequacy in Web UI testing frameworks for applications in diverse languages, respectively, while specialized techniques like in [178] improve client-side testing reliability.

Additionally, studies like in [179] emphasize on addressing security vulnerabilities in client-server setups; [180] on domain-specific data use for GUI testing; [181] related to code coverage measurement in industrial web applications; and [182] focusing on the use of evolutionary approaches addressing the gap in current black-box tools. Table A.18 provides a detailed description of the studies emphasizing white-box testing.

RQ10: Our survey reveals a predominant focus on black-box testing with 171 papers. On the other hand, only 22papers are dedicated to white-box testing. These two testing approaches are particularly used for automation in end-to-end, regression, and unit tests for web applications.

Table 8
Existing frameworks and studies utilizing them.

Category	Nr.Papers	List.papers
Selenium	118	[15,16,39–41,43,44,46,50–52,52,54–56,58,60–63,71,73–75,78–80, 86–91,95–97,106,107,115,120,121,124,130–132,134,135,137,140, 141,143–146,148,159–162,166,168,172,177,180–234]
Cypress	5	[92,117,142,163,235]
Puppeteer	1	[164]

Table 9
SBST/AI testing methods and studies utilizing them.

Category	# Papers	List papers
Search-based	21	[55,74,108,112,124,136,149,152,157–159,168,175,179,182,195,205,236–239]
Reinforcement learning	14	[15–17,110,138,142,147,151,154,155,165,167,186,240]
Computer vision	12	[46,51,61,66,100,139,183,189,190,222,241,242]
Other ML Models	11	[67,95,107,120,121,143,202,243–246]
Generative AI	11	[68,70,77,85,97,114,115,117,148,201,247]
Natural language processing	10	[42,63,64,119,126,130,192,215,219,248]
Reinforcement learning, Computer vision	1	[110]

5.11. RQ11 - Which existing frameworks and/or libraries are used to enable GUI testing?

We performed a full-text search on the surveyed papers to explore if the popular libraries for GUI testing are mentioned. These libraries - Selenium, Cypress, Puppeteer, and Playwright (recall Section 2.3) were used as search keywords.

It was to no surprise that Selenium is the most popular library. Among the 258 reviewed papers, Selenium is mentioned in 201 papers, Cypress in 23, Puppeteer in 10, and Playwright in 10 papers.

To examine how the libraries were referenced in the papers, a manual review was conducted on those containing the keywords. The review revealed that Selenium, Cypress, and Puppeteer were used in fewer papers than mentioned, while Playwright was only cited in the related work sections and not used in any of the studies. We therefore updated our dataset accordingly and regenerated the corresponding table to reflect these corrections. The role and usage of Selenium differ among the analyzed studies. Especially in studies focused on test case generation, like in [232] and [43], Selenium is the preferred library for the execution of the generated test cases. In studies like [46] and [186], Selenium is used as the basis upon which another tool is built. Selenium was used by the participants as the go-to tool to develop test scripts [87]. In [183] the authors make use of existing Selenium test suites to generate new test cases.

Puppeteer is used in [164] to automatically execute actions such as button clicks, and to build a wrapper to measure code coverage.

The role and usage of Selenium varies across the analyzed studies. In studies focused on test case generation, such as [232] and [43], Selenium is primarily used as the execution framework for the generated test cases. In other studies, including [46] and [186], Selenium serves as the foundation upon which additional tools or frameworks are built. In [87], Selenium is used by the participants as the primary tool for developing test scripts. Additionally, in [183], the authors leverage existing Selenium test suites to generate new test cases.

Puppeteer, by contrast, is employed in [164] to automatically execute user interactions, such as button clicks, and to implement a wrapper for measuring code coverage.

Table 8 presents a summary of studies categorized by the types of libraries used for testing.

RQ11: Our survey of current studies reveals that Selenium is the most widely used library for enabling GUI testing for web applications. The use of Selenium is considerably more prevalent compared to other libraries, Cypress and Puppeteer.

5.12. RQ12 - How many studies involve the use of AI techniques?

Of the 258 papers reviewed, 80 employed either search-based or AI-based software testing (SBST/AI) approaches. The automation of web testing has progressed from executing manually written test cases to also generating test cases automatically. Moreover, substantial research efforts have been made not only towards test generation, but also maintenance of tests, test case prioritization, etc, as described in Section 5.3. Considering the complexity of web applications, traditional testing methods face challenges in efficiently and effectively handling dynamic content and frequent updates.

These limitations have led to exploring AI-based solutions. Fig. 3 illustrates the publication rate of papers on the use of SBST/AI-based methods in web testing from January 2014 to May 2025. As observed from Fig. 3 the interest on AI techniques for web testing has been fairly constant since 2017, fluctuating between 5 and 11 papers per year. The last three years in particular indicate a rising interest in the field of web testing, as well as in the use of AI techniques within this domain. One-third of the papers published during

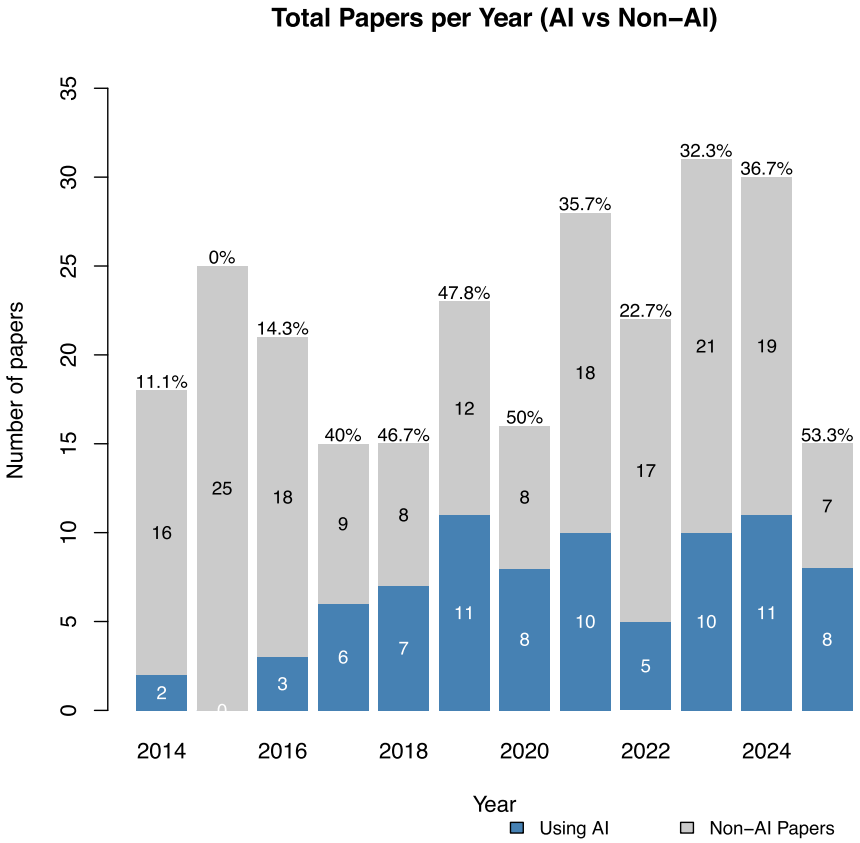


Fig. 3. Number of published papers using AI and not using AI.

this period employ AI techniques, a trend that may be linked to the rapid advancement of generative AI and large language models. The AI trend seems to be growing in 2025.

Furthermore, Table 9 presents a summary of studies categorized by various SBST/AI-based testing methods. As can be seen in Table 9, approximately one-quarter of the studies using SBST/AI-based methods employed search-based approaches, while the majority implemented various AI techniques, as outlined below.

5.12.1. Search-based approaches

Search-based approaches, particularly Genetic Algorithms, are extensively applied in web application testing for test case generation and UI testing. Studies focus on evolutionary algorithms to enhance client-server testing [168], DOM element crawling [195], and feature extraction [136]. Automated test generation integrates Java model simulations with search-based testing [238], while DOM selectors and XPath locators leverage genetic algorithms for UI testing [55]. Other studies explore path and input data generation [205], diversity-based generation [159], and parallel evolutionary testing [175]. Search-based regression testing and fail-safe behavior testing further expand these techniques [108,157,158]. Comprehensive end-to-end testing integrates search-based and machine learning solutions [74].

Search-based methods also play a critical role in vulnerability testing, particularly against XML injection attacks. Mutation-based test generation automatically exploits frontend vulnerabilities [236], while COMIX employs genetic algorithms for multi-vulnerability XML injection testing [179]. Input data generation for systems under test further ensures robustness in vulnerability detection [237].

Overall, search-based approaches including Genetic Algorithms, are pivotal in web testing, addressing test case generation, UI testing, and regression testing. In parallel, targeted methods efficiently identify vulnerabilities like XML injections, ensuring comprehensive application security [74,168,195,236,237].

5.12.2. AI-based approaches

Regarding AI-based approaches, the studies highlight extensive use of reinforcement learning (particularly Q-learning), computer vision, and natural language processing for testing, in that order. Additionally, various other machine learning approached are also employed using algorithms including neural networks. Details follow.

- Reinforcement learning. Reinforcement learning, particularly Q-learning, is extensively applied in web and UI testing, enabling automated test generation [15,17,167], prioritization [142], and coverage enhancement [16]. Studies explore curiosity-driven exploration [17], visual UI interaction [138], scriptless testing [154,249], and guided strategies for dynamic applications [16,240].

- Computer vision. Studies utilizing computer vision in web testing focus on UI analysis, automation, and visual segmentation. Approaches include feature mapping [183], widget classification [189], DOM migration via image recognition [61], and deep learning for UI testing [51,100]. Machine learning enhances visual testing [139,241], with CNNs aiding segmentation and classification [222].
- Natural language processing. Studies leveraging natural language processing (NLP) in web application testing focus on automating UI test case generation [42], improving model inference [119], prioritizing test cases [248], and exploratory testing using n-gram models [126]. NLP also aids in detecting dependencies [215], identifying web elements [63], repairing test suites [64], and enabling script-free test automation [219].
- Computer vision and reinforcement learning combined. Some studies, such as [110], also combine computer vision and reinforcement learning for automated GUI testing by extracting widgets, analyzing layouts, and representing pages as embedded states.

Additionally, the studies focus on various other machine learning (ML) approaches for web application testing, including UI test automation [95], test generation [192,243], and performance prediction [244]. Methods such as supervised learning [95], deep learning [201], generative adversarial networks (GAN) [201], and support vector machines (SVM) [120,121] are applied for tasks like UI element recognition [95], session-based testing [202], browser environment prioritization [243], and recommendation performance [107].

RQ12: Our survey of current studies reveals that a significant portion of papers, specifically 80, used SBST/AI approaches. About a quarter of these employing search-based methods, followed by Reinforcement Learning and Computer vision. AI techniques have been used not only for test generation, but also for performance prediction, test case prioratization etc. Research on SBST and AI in the context of web testing has remained relatively stable over time, with a noticeable increase in recent years

5.13. RQ13 - How many studies involve collaborations with industry?

As discussed in Section 5.4 for RQ4, there are 9 articles that we classified as "Experience Report" (recall Table 6), which are [80,194,200,210,238,250–253]. Furthermore, as discussed in Section 5.5 for RQ5, there are 31 articles involving algorithm/tool experiments on industrial systems. The number of those industrial systems in each study ranges from 1 to 20 (recall Table 7). When it comes to studies with human subjects (Section 5.6 for RQ6), out of 34 there are 5 that involved industrial systems [47,77,126,134,147]. The work in [15] can be considered a "tool paper", where the different features and success stories throughout the years of the GUI (including web) testing tool TESTAR are described. This includes listing all the different studies in industry where TESTAR was used. In total, there are 46 articles in which some form of academia-industry collaboration or industry report was presented, which is 17.8% of the total articles reviewed in this survey. Table A.19 lists these 46 articles.

This data shows a promising trend in this research field, where software engineering results are starting to become usable and able to scale to real-world applications. This is in contrast with what reported in [34], where it was stated "*Most papers have little industrial relevance, since industry is not involved in most studies*", as only 6 articles in their survey involved web applications from industry. For an engineering field, the technology transfer from academic research to industrial practice is commendable. Due to their importance, in the reminder of this section we will discuss these 9experience reports in more details. The work in [250] discussed the experience of using the Automated Testing Framework (ATF) on two clients of PrimeUp,[13] an IT Services and IT Consulting company in Brazil. The addressed problem was: "*Our company faces a challenge where each client requires different technologies to be employed by our test team to adapt to their needs and applications. Thus, our testers must learn different technologies and tools for each client*" [250]. To address these challenges, "*ATF was created by PrimeUp in partnership with Universidade Federal Fluminense …which provides a standardized and organized approach for developing end-to-end tests for various devices and browsers*" [250].

The work in [251] presents the "*…experience in developing and introducing a set of large automated test suites (more than 50 KLOC in total), using best practices in state-of-the art and -practice …*". The study was done at Innova,[14] a large software firm in Turkey. It was stated that: "*Test execution effort (running all test cases) which used to take 2 days was reduced to 1 hour only …our industry-academia collaboration and joint work were indeed a success story*" [251].

The article [194] was authored by four employees at Varidesk,[15] a USA-based company that sells office furniture. That work "*discusses our efforts at Varidesk to automate web tests against our main website*" [194]. One major contribution of that work is that "*Recognizing that not enough research has been published on what it takes to bring in an existing/create a new automation framework, we candidly talk about the requirements we needed to meet, and how we ultimately went about meeting them*" [194].

WebMate was presented in [200], where the transfer from academic research to a startup in Germany (Testfabrik[16]) is discussed. WebMate is a tool aimed at web testing for detecting cross-browser issues. The article discusses the challenges and lessons learned in creating a commercial product out of academic research in web testing.

[13] https://www.primeup.com.br/
[14] https://www.innova.com.tr
[15] https://www.vari.com/
[16] https://testfabrik.com/en/

The article [238] was written by one of us (Prof. Arcuri), where "*I report on my direct experience as a PhD/post-doc working in software engineering research projects, and then spending the following five years as an engineer in two different companies*" [238]. Both these companies are located in Norway (the multi-national Schlumberger[17] and Telenor[18]). This reported experience included the discussion of the challenges of manually writing web tests with Selenium and WireMock. An argument was made that, during that time (2011–2016), "*research in software engineering, and particularly software testing, has had only limited impact on current practices in industry*" [238].

The work in [252] presents three case studies in industry, for risk-based testing. The three involved enterprises are not named in the article. Some of the involved systems are large web applications with millions of lines of code.

The work in [253] discusses " *the challenges that we faced in the early stages of a technology transfer project aimed to introduce a test generator in a software house that develops Web-based Enterprise Resource Planning (ERP) solutions*". The involved enterprise is not named, and it is described as "*a mid-size Italian software house*" [253]. Different challenges are discussed in the article, including regarding scalability, lack of oracles and test reporting.

Testinium[19] is a large software testing company in Turkey. In the work in [210], they report on their use of the Gauge[20] framework in several of their clients. Gauge is an open-source automated testing framework for behavior-driven development for web applications.

As discussed in Section 5.3, flakiness in web test cases is a major source of problems. In [80], its authors "*were involved in the refactoring of an existing automated flaky E2E test suite for a large Web application. In this paper, we report on our experience …Our procedure allowed to reduce the flakiness to virtually zero*". The involved organization was PRINTO,[21] a non-profit international research medical network.

RQ13: Out of 258 surveyed articles, there are 31 algorithm/tool studies using industrial systems, 5 studies with human subjects on industrial systems, 1 tool paper and 9 experience reports in industry. Although 17.8% still represents a minority of the studies, this shows a promising, positive trend in the research field of web testing.

6. Discussion and future research agenda

The large number of publications (i.e., 258) in the period of 2014–2025 shows the importance of the topic of web testing in the research community. Many of these articles were published in the leading conferences and journals in software engineering research, like for example FSE and ICSE, and not just specialized venues for software testing like ICST (RQ1). This provides evidence on the maturity and quality of research work done on this topic.

The field of web testing covers several different aspects, although test case generation is the most common (RQ3). This is an important complex problem that is still not fully solved, with scalable solutions that are widely adopted in industry.

When doing research, there are different kinds of studies that can be carried out. In the context of web testing, by far the most common case is to design and present a novel technique, and evaluate in the lab on a set of web applications (RQ4). Other kinds of studies, like for example with human subjects or analyses of existing practices, are comparatively fewer. This creates a large gap in the literature that needs to be filled.

Most studies in the literature use open-source projects as artifacts for the empirical evaluations (RQ5). Although there is a positive trend in using systems from industrial collaborations in the studies, the number of used applications is typically small. This creates major threats to the external validity of many studies in the literature. Unfortunately, setting up and configuring real-world web applications for empirical studies is no trivial task. There is a need and an opportunity here to create easy-to-reuse collections of web applications to foster and simplify future studies in web testing.

As already mentioned, studies with human subjects are just a few, and several of those only involve students (RQ6). This kind of studies are expensive, time-consuming and risky to carry out, especially when done in industry with industrial practitioners, which explains their rarity. However, they are essential to study how techniques only used in the lab by their authors can actually be employed in the real-world.

The large majority of studies involve designing novel techniques. These techniques are often implemented in software tools, but only 1/3 of them seems released as open-source (RQ7). This is a major issue, as it can hinder comparisons with existing work (RQ8). Comparisons with the state-of-the-art is essential to properly evaluate newly proposed techniques. Furthermore, unless commercial web testing tools integrate academic techniques evaluated in research papers, the impact of these latter on industrial practice would be limited. Even in the case of open-source tools, these are often used in just a few studies (typically only one), and then discontinued by their academic authors (e.g., PhD students graduating and moving on to different research topics). Mature academic open-source

[17] https://www.slb.com/
[18] https://www.telenor.no
[19] https://testinium.com/
[20] https://gauge.org/index.html
[21] https://www.printo.it/

tools that are developed throughout the years, used in several different studies, and that can be used by practitioners in industry, seems a rarity. A notable exception is TESTAR.

Regarding impact, even if academic tools are not directly used by practitioners in industry (e.g., due to usability concerns and lack of maintenance), automatically finding and reporting faults in existing systems can provide concrete benefits (RQ9). However, test case generation is only one problem in the research field of web testing. How to increase the industry impact of other kinds of studies is an important problem to address.

Testing a web application from its GUI frontend requires using a browser and interacting with HTML pages. However, the behavior of the application strongly depend on the software that is run on the frontend (e.g., JavaScript event handlers) and what running in the backend (including databases). However, very few techniques and studies in the literature exploit the knowledge of the source code when testing a web application (RQ10), in the so-called white-box testing. As the analysis of the source code of an application can lead to better testing results (e.g., considering the literature of white-box testing in other domains besides web testing), this creates a major research opportunity for designing novel techniques that can further push forward the state-of-the-art.

To interact with a browser in a test case, specialized libraries are needed. Those are Selenium, Cypress, Puppeteer, and Playwright. However, in academic studies, Selenium is by the far the most used, whereas the other libraries are nearly ignored (RQ11). Considering the widespread use of these other libraries in industry, this might hinder technology transfer from academic results to industrial practice. Why academics focus only on Selenium, and whether the choice of a library is just a technical detail and not relevant for research work, is something that would need more investigation.

Testing web applications is an important topic that has been investigated for at least the last three decades. It is composed of many complex problems, from test generation to test maintenance. The use of AI techniques might provide working solutions for several of these challenging research problems. As such, the use of AI techniques has seen a major increase in the academic literature in the last few years (RQ12). How to best use these techniques, and how to combine them (e.g., SBST + LLM) to get better results, are important research topics of high potential, especially when applied in contexts that have seen only little work in the literature so far, like white-box testing (RQ10).

Research in a practical engineering field such as software engineering should, in the long term, be applicable and scalable to real-world applications and contexts. Direct applications of academic results in industry via academia-industry collaborations can help reaching such goals. In our survey, we have seen that 17.8% of the reviewed articles involved some sort application in industry (RQ13). This shows a positive, inspiring trend. The use of innovative techniques like AI (RQ12) might be the key to solve real-world problems that can be of value for practitioners in industry (RQ13).

Based on the findings of our study, we outline several directions for future research aimed at advancing the state of the art in web testing. While substantial progress has been made over the past decade, our results indicate that several challenges remain open and warrant further investigation.

First, test case generation remains the most extensively studied problem, yet still not fully solved. Future work should continue to explore more effective, scalable, and robust techniques, particularly for complex and dynamic web applications. White-box techniques in particular are far from being explored and their potential fully unlocked.

Second, the rapid advancement of artificial intelligence techniques, especially large language models (LLMs), opens new opportunities for web testing. Future research should investigate how these models can be systematically integrated into different stages of the testing process and rigorously evaluated against existing approaches.

Third, given the high practical relevance of web testing for industry, stronger collaboration between academia and industry is essential. Although some industrial involvement has been reported, such studies remain limited. Future research should emphasize more industrial evaluations and real-world case studies to improve technology transfer and assess the applicability of academic solutions in practice.

Fourth, there have been efforts to construct comprehensive datasets of open-source repositories that can serve as benchmarks for end-to-end web testing. Notable examples include E2EGit [93] and WebEv [92].

Such benchmarking initiatives are crucial for enabling systematic and reproducible evaluation of testing tools and techniques, as they provide a common set of systems under test and facilitate fair comparison across studies. By relying on shared benchmarks, researchers can more accurately assess progress in the field, improve result comparability, and reduce the reliance on ad hoc experimental setups.

Finally, our analysis highlights a lack of uniformity in how tool performance and fault detection are reported. This variability hinders meaningful comparison across studies. As a result, future work should focus on establishing standardized benchmarks, common fault taxonomies, and shared experimental frameworks to improve reproducibility and comparability in web testing research.

7. Threats to validity

To ensure the validity of our findings, we have taken various measures to minimize potential biases. However, some limitations remain, which we discuss in this section. A potential limitation of this study is the paper selection process. We conducted searches in the most widely used academic databases, with search queries carefully reviewed and agreed upon by all authors. Additionally, we performed both forward and backward snowballing to identify further relevant studies. Since this process was manually conducted, we implemented a validation step where at least two authors reviewed the final list of selected papers to minimize errors and reduce the risk of omitting relevant studies. However, despite these efforts, there remains a possibility that some relevant papers on web

testing may have been missed. Additionally, there remains a possibility that other researchers may obtain different results in the selection of relevant papers, leading to variations in the final list of included studies.

A second potential limitation of this study is the data extraction process. While data analysis was conducted rigorously, all manually extracted data was verified by at least two authors. In cases of disagreement, additional authors reviewed the paper, and the final extracted data was determined through consensus. Certain qualitative data points, such as category and contribution, were defined by the authors. Although we have taken great care to ensure accuracy in assigning labels and definitions, there remains the possibility of misinterpretation. Additionally, other researchers may have differing perspectives on these classifications.

Third, we did not conduct a formal quality assessment of the selected articles for several reasons. Our primary goal was to provide a broad and comprehensive overview of the current state of the art in web testing, rather than to evaluate or rank individual studies. Because our corpus was collected entirely from well-established academic databases and reputable publication venues, we already operated under the assumption that the included works met a baseline level of scholarly quality. Conducting an additional quality assessment on our side would have introduced a degree of subjectivity, potentially affecting the neutrality of our review.

Moreover, applying quality assessment criteria carries an inherent risk of bias, as different research methodologies, publication types, and evaluation standards might be unfairly weighted. Attempting to impose uniform criteria could therefore distort the representation of the field.

To avoid these potential biases, we opted not to perform a separate quality assessment, focusing instead on capturing the full picture of research activity and trends over the past decade.

8. Conclusions

In this study, we analyzed 258 papers published over the past decade (2014–2025), focusing on web testing. We searched through major databases and reviewed every paper to ensure its relevance. Our research questions were designed to provide a comprehensive and detailed understanding of the trends and current state of web testing.

The volume of publications indicates an interest from the research community, with annual publication ranging from 15 to 31 papers across various conferences and journals, notably ICST. These studies cover a broad range of topics and study categories. While both black-box and white-box testing are utilized in research, black-box testing is significantly more prevalent. Additionally, we examined trends in the adoption of AI techniques. Our findings also highlight the development of numerous web testing tools, some of which have been released as open-source solutions. Furthermore, many of these studies include empirical evaluations, ranging from algorithmic experiments to human participant studies, or a combination of both. The experiments were primarily conducted in academic settings. However, there is a growing trend of collaboration with industry.

This survey offers valuable insights into the evolution of web testing, key trends, challenges, and advancements in the field. Future research can build on these findings, expanding open-source availability of web testing tools, to further refine testing methodologies and explore emerging technologies that enhance the efficiency and effectiveness of web application testing.

CRediT authorship contribution statement

Iva Kertusha: Writing – review & editing, Writing – original draft, Validation, Software, Methodology, Formal analysis, Data curation, Conceptualization; **Gebremariam Assres:** Writing – review & editing, Writing – original draft, Validation, Formal analysis, Data curation; **Onur Duman:** Validation, Methodology, Data curation; **Andrea Arcuri:** Writing – review & editing, Writing – original draft, Validation, Supervision, Methodology, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work is funded by the European Research Council (ERC) under the European Union's Horizon 2020 research and innovation programme (EAST project, grant agreement No. 864972).

Appendix A. Supplementary tables

Table A.10
Common publishing venues.

Venue	Nr.Papers	Venue.Type
Other conferences	66	conference
ICST	28	conference
Other journals	24	Journal
FSE	11	conference
ICSE	10	conference
ICSTW	10	workshop; conference
JSS	10	journal
STVR	8	journal
TSE	7	journal
ASE	6	conference
QUATIC	6	conference
SQJ	6	journal
AST	5	conference; workshop
EMSE	5	journal
ISSTA	5	conference
IST	5	journal
ICSME	4	conference
TOSEM	4	journal
COMPSAC	3	conference
ICWE	3	conference
<Programming>	2	conference
AUSE	2	journal
DSA	2	conference
ESEM	2	conference
ICEBE	2	conference
IEEE Access	2	journal
IJSEKE	2	journal
ISMSIT	2	conference
ISSRE	2	conference
JSEP	2	journal
SoSyM	2	journal
SSBSE	2	conference

Table A.11
List of categories and respective papers.

Category	Nr.Papers	List.papers
Test Generation	90	[14,15,17,95,97,102,103,107–110,110,111,114,117,120,121,123,124,130,132–135,147,148,152,154,155,157,158,160,161,163–165,167,168,171,172,175,176,178–180,182,183,185–187,189,198,199,201–204,212,222–225,227,230–232,236–240,244,253–270]
Test Maintenance	47	[42–50,52,52–65,67–70,72–83,85–91]
Testing Framework	43	[39,40,96,98,101,105,106,129,139–141,143–145,162,191,194,196,207,210,213,215–221,226,229,233–235,247,250–252,271–276]
Test Generation; Model Inference	18	[13,16,126,136–138,159,188,192,197,205,208,245,277–281]
Oracle	10	[100,127,131,166,190,193,206,211,241,282]
Coverage Criteria	8	[150,169,174,181,214,283–285]
Model Inference	7	[119,153,184,242,246,286,287]
Test Prioritization	5	[99,142,243,248,288]
Test Generation; Model Inference; Oracle	3	[195,209,289]
Test Generation; Coverage Criteria	3	[116,151,173]
Model Inference; Oracle	2	[104,139]
Dataset	2	[92,93]
Test Generation; Oracle	2	[200,228]
Test Generation; Test Maintenance	2	[51,66]
Testing Framework; Test Generation	1	[146]
Testing Framework; Oracle	1	[125]
Test Generation; Model Inference; Coverage Criteria	1	[128]
Taxonomy; Test Maintenance	1	[71]
Taxonomy; Oracle	1	[265]
Taxonomy	1	[290]
Test Generation; Testing Framework	1	[291]
Test Generation; Testing Framework; Model Inference	1	[292]
Testing Framework; Oracle	1	[293]
Test Maintenance; Model Inference	1	[41]
Coverage Criteria; Taxonomy	1	[177]
Test Prioritization; Coverage Criteria	1	[112]
Model Inference; Oracle	1	[113]
Test Generation; Test Prioritization	1	[149]
Model Inference; Test Maintenance	1	[84]
Test Generation	1	[115]

Table A.12
List of papers for every contribution.

Type	Nr.Papers	List.papers
NToT;AE	161	[14,16,17,39,41,42,44–46,49,51–64,67–70,72,75,79,83,85,86,88,90,91,95–97,99,100,102–104,107–113,115–117,119–121,150–153,155,157–169,171–176,178–180,183,185,186,188–190,192,193,195,197,199,201–203,205,206,209,211–218,221–228,231,232,234,236,237,239–241,243–248,254–256,258,260,262–265,265,268–271,275,278–282,284–289,292,293]
PP	20	[13,15,43,65,66,74,84,89,92,93,154,181,182,184,220,235,267,272,276,291]
NToT	17	[78,139,187,196,198,207,208,229,230,233,257,259,261,266,274,277,283]
NToT;HE	13	[47,48,50,124–126,128,129,133,137,140,141,146]
HE	10	[87,123,127,130–132,134,139,145,147]
AE	10	[52,98,101,105,106,114,191,219,242,273]
ER	9	[80,194,200,210,238,250–253]
NToT;AE;HE	7	[77,135,136,138,142,148,149]
OSA	5	[71,73,81,82,290]
survey	3	[40,76,143]
AE;OSA	1	[204]
NToT;survey	1	[144]
NToT;AE;OSA	1	[177]

Table A.13
Number of participants.

Category	Ref	# academic	# practitioners	# total
Oracle	[127]	24	2	26
Oracle	[131]	0	1	1
Test Generation	[132]	20	0	20
Test Generation	[123]	0	not specified	not specified
Test Generation	[133]	0	4	4
Test Generation	[134]	0	6	6
Test Generation	[124]	not specified	0	not specified
Test Generation	[130]	1	3	4
Test Generation	[135]	5	0	5
Test Generation	[147]	0	2	2
Test Generation	[148]	5	0	5
Test Generation; Model Inference	[138]	0	3	3
Test Generation; Model Inference	[126]	39	8	47
Test Generation; Model Inference	[136]	0	5	5
Test Generation; Model Inference	[137]	18	0	18
Test Generation; Model Inference; Coverage Criteria	[128]	1	2	3
Test Generation; Test Prioritization	[149]	0	5	5
Test Maintenance	[47]	0	1	1
Test Maintenance	[48]	6	0	6
Test Maintenance	[50]	0	3	3
Test Maintenance	[87]	36	0	36
Test Maintenance	[77]	0	12	12
Test Maintenance	[76]	0	94	94
Test Prioritization	[142]	28	0	28
Testing Framework	[139]	144	0	144
Testing Framework	[129]	49	32	81
Testing Framework	[145]	25	0	25
Testing Framework	[141]	4	0	4
Testing Framework	[140]	6	0	6
Testing Framework	[40]	0	78	78
Testing Framework	[143]	0	72	72
Testing Framework	[144]	0	148	148
Testing Framework; Oracle	[125]	8	2	10
Testing Framework; Test Generation	[146]	0	20	20

Table A.14
Systems under test (SUTs) and associated papers.

SUT	Nr. of.Papers	Papers
addressbook	34	[16,39,44,54,56–58,60–62,64,70,71,75,79,87,88,91,112,119,124,149,161,166,168,175,177,205,214–217,284,289]
claroline	22	[16,39,44,47,54–58,61,64,70,88,112,119,166,177,212,214–216,289]
petclinic	20	[17,42,69,88,90,113,119,131,135,148,159,163,164,167,185,186,223,246,263,286]
mantisbt	18	[43,49,54,56–58,63,64,70,119,130,180,215,216,227,240,286,289]
mrbs	15	[39,49,54,56–58,61,70,75,119,177,183,215,216,286]
ppma	15	[39,44,54,56,58,61,62,79,88,112,119,169,215,216,289]
dimeshift	13	[17,42,112,119,135,148,159,164,167,246,263,286,289]
collabtive	12	[16,44,54,56–58,64,70,79,85,215,216]
splittypie	10	[17,135,148,159,164,167,186,246,263,286]
pagekit	9	[17,42,112,119,159,164,246,286,289]
retroboard	8	[17,119,135,159,164,167,246,263]
phonix	8	[112,119,135,148,159,246,263,282]
schoolmate	8	[16,75,149,168,175,214,284,285]
Joomla	7	[49,59,60,63,71,85,130]
TuDu	6	[86,124,161,173,174,177]
shopizer	6	[39,51,90,131,153,219]
dolibarr	5	[59,60,71,180,286]
FAQForge	5	[149,168,175,227,285]
timeclock	5	[16,75,214,284,285]
phoenix-trello	4	[17,164,167,289]
phormer	4	[62,166,214,254]
parabank	3	[111,153,185]
PHP-Fusion	3	[49,60,71]
YourContacts	3	[60,71,75]
wolfCMS	3	[166,212,214]
brotherhood	3	[16,166,214]
tinymce	3	[62,161,173]
expresscart	3	[87,130,219]
moodle	3	[59,112,245]
tikiwiki	3	[56,61,177]
odoo	3	[151,152,155]
phpaacms	2	[149,175]
OIM	2	[130,219]
OsCommerce	2	[227,284]
Wordpress	2	[149,284]
mediawiki	2	[85,137]
PrestaShop	2	[51,130]
bookstore	2	[86,281]
Classifieds	2	[86,281]
samegame	2	[173,174]
tunnel	2	[173,174]
ghostbusters	2	[173,174]
simplecart	2	[173,174]
wymeditor	2	[173,174]
photo-gallery	2	[161,212]
enterpriseStore	2	[166,212]
phpagenda	2	[60,71]
myCollaboration	2	[60,71]
MyMovieLibrary	2	[62,71]
jpetstore	1	[244]

Table A.15
Introduced or improved tools.

Tool	Open Source	# Papers	List papers
UNNAMED	No	61	[48,50–55,67,72,83,85,95,99,100,103,107,112,119–121,125,126,136,138,139,139,149,162,168,172,175,183,186,189,190,192,194,196,198,199,201–203,207–209,213,214,221–225,236,237,241,245,258,260,274,275]
UNNAMED	Yes	9	[58,63,70,97,195,232,243,271,282]
TESTAR	Yes	8	[15,111,147,151–155]
Apogen	Yes	4	[41,88,89,91]
FSMWeb	No	3	[108,157,158]
Not implemented Yet	NA	3	[132,133,150]
Cornpickle	Yes	2	[265],P246
E2E-LOADER	Yes	2	[268,270]
PESTO	Yes	2	[39,61]
PIRLTest	Yes	2	[110,110]
ViMoTest	Yes	2	[267,269]
Vista	Yes	2	[43,44]
WAPG	No	2	[116,128]
X-BROT	No	2	[193,211]
AJAS	No	1	[277]
ApiCarv	Yes	1	[185]
Assessor	Yes	1	[90]
ATA	No	1	[86]
ATRINA	Yes	1	[166]
Automation Xtreme	No	1	[233]
autoOrchTest	No	1	[259]
AWET	Yes	1	[135]
COLOR	No	1	[49]
COMIX	No	1	[179]
ComJaxTest	No	1	[279]
Confix	No	1	[254]
CrawLabel	Yes	1	[42]
Crawljax	No	1	[14]
Cypress Copilot	Yes	1	[117]
Cytestion	Yes	1	[163]
DANTE	Yes	1	[263]
DBInputs	Yes	1	[180]
DIG	Yes	1	[159]
eBat	Yes	1	[164]
EventBreak	No	1	[255]
EvoMaster	Yes	1	[182]
FOREPOST	No	1	[244]
FraGen	Yes	1	[289]
Fraunhofer Tool	No	1	[197]
FRET	No	1	[234]
FsCheck	No	1	[280]
FuzzE	Yes	1	[231]
GIPGUT	Yes	1	[141]
GTpql	Yes	1	[240]
GUITAR	Yes	1	[13]
history-diagnostics	Yes	1	[257]
InwertGen	No	1	[161]
jFAT	No	1	[226]
JSdep	Yes	1	[262]
JSEFT	Yes	1	[173]
Judge	Yes	1	[246]
KeyDriver	No	1	[261]
KeyjaxTest	No	1	[278]
Kraken 2.0	Yes	1	[291]
Link	No	1	[256]
LOTUS	No	1	[228]
MAEWU	Yes	1	[177]
MAJD	Yes	1	[146]
MARG	No	1	[165]
MBTCover	Yes	1	[181]
MBUITC	Yes	1	[292]
MoLeWe	Yes	1	[137]
Morpheus Web Testing	Yes	1	[188]
Mutandis	Yes	1	[174]

Table A.15
Continue.

Tool	Open Source	# Papers	List papers
OOGSSA	No	1	[239]
P-GUI	No	1	[96]
PARADIGM-ME	No	1	[124]
PARTE	No	1	[227]
PathFinder	Yes	1	[247]
Pattern-Based Usability Testing	No	1	[293]
QExplore	Yes	1	[16]
RClassify	No	1	[178]
ReDeCheck	No	1	[206]
RLTCP	No	1	[142]
Robot (framework)	No	1	[187]
ROBULA+	Yes	1	[56]
Scout plugin	No	1	[140]
Scout-LLM	Yes	1	[77]
ScPlay, ScRec, ScEdit	No	1	[129]
Selenium-Jupiter	Yes	1	[218]
SEMTER	No	1	[64]
SgaTest	No	1	[113]
Sibilla	No	1	[286]
SideeX	No	1	[78]
SIDEREAL	No	1	[62]
Similo	Yes	1	[45]
SkyFire	Yes	1	[264]
SleepReplacer	Yes	1	[79]
StackFul	No	1	[171]
Stile	Yes	1	[216]
Subweb	No	1	[205]
SymJS	No	1	[176]
Tansuo	No	1	[281]
TEDD	Yes	1	[215]
TERMINATOR	No	1	[248]
TesMa	No	1	[102]
TEST-Web	Yes	1	[229]
Testilizer	Yes	1	[212]
TOM	Yes	1	[160]
TransCompiledCodeCoverage	Yes	1	[169]
UAES	No	1	[69]
UITESTFIX	No	1	[57]
V-DOM	No	1	[285]
VETL	Yes	1	[148]
VON Similo LLM	Yes	1	[68]
Waterfall	No	1	[60]
WebDriverManager	Yes	1	[144]
WebEvo	Yes	1	[46]
WebExplor	No	1	[17]
WebMate	No	1	[200]
WEBMOLE	Yes	1	[287]
WebQT	No	1	[167]
WebRR	No	1	[47]
WebTest	Yes	1	[284]
WebTestRepair	No	1	[75]
WebTestSuiteRepair	No	1	[59]

Table A.16
List of tools used for comparison.

Tool	# Papers	List papers
Crawljax	10	[16,17,135,188,205,263,279,285,287,289]
WebExplor	8	[16,17,110,110,148,164,165,167]
WATER	8	[43,44,46,57,58,60,64,70]
Vista	4	[46,47,57,64]
Selenium	3	[47,53,219]
Artemis	3	[173,176,262]
WebEvo	3	[57,64,70]
Subweb	2	[17,159]
Robula+	2	[53,62]
Atusa	2	[159,263]
SFTM	2	[57,70]
TESTAR	1	[163]
LML	1	[45]
DIG	1	[17]
Random	1	[17]
WATERFALL	1	[46]
Atlassian Clover	1	[169]
EclEmma	1	[169]
monkey testing	1	[97]
GPT-3.5-Turbo	1	[97]
WebQTse	1	[167]
WebQTr	1	[167]
MUTANDIS	1	[166]
FeedEx	1	[278]
FireCrystal	1	[136]
FireCrow	1	[136]
vMOSA	1	[179]
MBUITC	1	[292]
VeriWeb	1	[279]
Web 1.0 Crawler	1	[287]
Depth-First Crawler	1	[287]
Tansuo	1	[287]
WebExplorer	1	[16]
Montoto	1	[62]
Selenium WebDriver	1	[219]
Selenium IDE	1	[219]
unnamed commercial	1	[219]
APOGEN	1	[223]
DANTE	1	[135]
WebSphinx	1	[281]
Link Checker Pro–	1	[281]
Axe DevTools Pro–	1	[228]
EVENTRACER	1	[178]
R4	1	[178]
Monkey	1	[110]
QExplore	1	[165]
Katalon	1	[52]
LINK	1	[180]
KeyjaxTest	1	[113]
WebEmbed	1	[246]
GraphMAE	1	[246]
ContentHash	1	[246]
Levenshtein	1	[246]
RTED	1	[246]
SimHash	1	[246]
BlockHash	1	[246]
Pdiff	1	[246]
Phash	1	[246]
Histogram	1	[246]
SIFT	1	[246]
SSIM	1	[246]
FragGen	1	[246]
VISTA	1	[70]
EDIT DIS	1	[70]
SleepReplacer	1	[85]

Table A.17
Report number of faults.

Ref	Title	Category	# Reported Faults
[159]	Diversity-Based Web Test Generation	Test Generation; Model Inference	3
[155]	Automated Testing of Web Applications with TESTAR: Lessons Learned Testing the Odoo Tool	Test Generation	5
[255]	EventBreak: analyzing the responsiveness of user interfaces through performance-guided test generation	Test Generation	6
[204]	Test them all, is it worth it? Assessing configuration sampling on the JHipster Web development stack	Test Generation	6
[13]	GUITAR: an innovative tool for automated testing of GUI-driven software	Test Generation; Model Inference	7
[280]	Property-based testing of web services by deriving properties from business-rule models	Test Generation; Model Inference	8
[147]	Evaluating the TESTAR tool in an Industrial Case Study	Test Generation	11
[134]	Scripted and scriptless GUI testing for web applications: An industrial case	Test Generation	12
[151]	Q-learning strategies for action selection in the TESTAR automated testing tool	Test Generation; Coverage Criteria	12
[152]	Using genetic programming to evolve action selection rules in traversal-based automated software testing: results obtained with the TESTAR tool	Test Generation	13
[110]	Effective, Platform-Independent GUI Testing via Image Embedding and Reinforcement Learning	Test Generation	13
[164]	eBAT: An Efficient Automated Web Application Testing Approach Based on Tester's Behavior	Test Generation	17.8
[279]	Automated Testing of Web Applications Using Combinatorial Strategies	Test Generation; Model Inference	18
[99]	A Method for Test Cases Reduction in Web Application Testing Based on User Session	Test Prioritization	20
[209]	Input Contract Testing of Graphical User Interfaces	Test Generation; Model Inference; Oracle	25
[168]	Test Case Generation Based on Client-Server of Web Applications by Memetic Algorithm	Test Generation	26
[171]	Prioritising Server Side Reachability via Inter-process Concolic Testing	Test Generation	26
[173]	JSEFT: Automated JavaScript Unit Test Generation	Test Generation; Coverage Criteria	35
[273]	Behavior-driven development (BDD) Cucumber Katalon for Automation GUI testing case CURA and Swag Labs	Testing Framework	36
[128]	Using Petri Nets to Test Concurrent Behavior of Web Applications	Test Generation; Model Inference; Coverage Criteria	36
[224]	Website Functionality Testing Using the Automation Tool	Test Generation	36
[165]	Can Cooperative Multi-Agent Reinforcement Learning Boost Automatic Web Testing? An Exploratory Study	Test Generation	39.5
[231]	FuzzE, Development of a Fuzzing Approach for Odoo's Tours Integration Testing Plateform	Test Generation	45
[116]	Testing concurrent user behavior of synchronous web applications with Petri nets	Test Generation; Coverage Criteria	57
[167]	A Reinforcement Learning Approach to Generating Test Cases for Web Applications	Test Generation	69
[16]	QExplore: An exploration strategy for dynamic web applications using guided search	Test Generation; Model Inference	69
[166]	Atrina: Inferring Unit Oracles from GUI Test Cases	Oracle	76
[149]	Multi-Objective Test Case Generation for Web Applications with Limited Resources	Test Generation; Test Prioritization	80
[234]	Tree-Based Synthesis of Web Test Sequences from Manual Actions	Testing Framework	86
[265]	Testing Web Applications Through Layout Constraints	Taxonomy; Oracle	90
[275]	Event-driven web application testing based on model-based mutation testing	Testing Framework	92
[110]	Effective, Platform-Independent GUI Testing via Image Embedding and Reinforcement Learning	Test Generation	128
[163]	Cytestion: Automated GUI Testing for Web Applications	Test Generation	142
[124]	Pattern-based GUI testing: Bridging the gap between design and quality assurance	Test Generation	148
[127]	Empirical validation of an automatic usability evaluation method	Oracle	195
[162]	A Selenium-based Web Application Automation Test Framework	Testing Framework	255
[161]	Incremental Web Application Testing Using Page Object	Test Generation	569
[258]	Combinatorial Testing of Full Text Search in Web Applications	Test Generation	797
[160]	TOM: a Model-Based GUI Testing framework	Test Generation	935
[71]	Why do Record/Replay Tests of Web Applications Break?	Taxonomy; Test Maintenance	1065
[148]	Leveraging Large Vision Language Model For Better Automatic Web GUI Testing	Test Generation	1780
[17]	Automatic Web Testing Using Curiosity-Driven Reinforcement Learning	Test Generation	3478
[123]	Evaluation for Web GUI Automation Testing Tool - Experiment	Test Generation	not specified
[125]	Gamified Exploratory GUI Testing of Web Applications: a Preliminary Evaluation	Testing Framework; Oracle	not specified
[98]	Test Driven Development in Action: Case Study of a Cross-Platform Web Application	Testing Framework	not specified
[138]	Automating GUI Testing with Image-Based Deep Reinforcement Learning	Test Generation; Model Inference	not specified
[236]	Automatic Generation of Tests to Exploit XML Injection Vulnerabilities in Web Applications	Test Generation	not specified
[197]	Model generation to support model-based testing applied on the NASA DAT Web-application - An experience report	Test Generation; Model Inference	not specified
[274]	A framework for testing web applications using action word based testing	Testing Framework	not specified
[260]	Syntax-based test case generation for web application	Test Generation	not specified
[261]	A Keyword-Driven Tool for Testing Web Applications (KeyDriver)	Test Generation	not specified
[200]	WebMate: Web Application Test Generation in the Real World	Test Generation; Oracle	not specified

Table A.17
(Continued).

Ref	Title	Category	# Reported Faults
[51]	Using deep learning for selenium web UI functional tests: A case-study with e-commerce applications	Test Generation; Test Maintenance	not specified
[15]	testar - scriptless testing through graphical user interface	Test Generation	not specified
[136]	Test-Driven Feature Extraction of Web Components	Test Generation; Model Inference	not specified
[137]	Model-based testing leveraged for automated web tests	Test Generation; Model Inference	not specified
[292]	Automated Model-Based Test Case Generation for Web User Interfaces (WUI) From Interaction Flow Modeling Language (IFML) Models	Test Generation; Testing Framework; Model Inference	not specified
[284]	Exploring output-based coverage for testing PHP web applications	Coverage Criteria	not specified
[241]	Artificial intelligence in automated system for web-interfaces visual testing	Oracle	not specified
[225]	A Semantic Web Enabled Approach to Automate Test Script Generation for Web Applications	Test Generation	not specified
[264]	Skyfire: Model-Based Testing with Cucumber	Test Generation	not specified
[265]	Testing Web Applications Through Layout Constraints	Test Generation	not specified
[112]	Segment-Based Test Case Prioritization: A Multi-objective Approach	Test Prioritization; Coverage Criteria	not specified
[239]	A Hybrid Approach for Automated GUI Testing using Quasi-Oppositional Genetic Sparrow Search Algorithm	Test Generation	not specified
[229]	TEST-Web: An Automated Web Application Testing Tool for Enhanced Efficiency and Consistency	Testing Framework	not specified
[288]	A Regression Test Case Prioritization Technique for Web Application Using User Session Data	Test Prioritization	not specified
[267]	ViMoTest: A Tool to Specify ViewModel-Based GUI Test Scenarios using Projectional Editing	Test Generation	not specified
[140]	Conceptualization of Multi-user Collaborative GUI-Testing for Web Applications	Testing Framework	not specified

Table A.18
RQ11 - List of studies using white-box testing techniques with highlights.

Citation	Title	Study.highlights
[169]	Code coverage for any kind of test in any kind of transcompiled cross-platform applications	Presents a framework-independent Java tool for calculating code coverage for cross-platform applications. The tool offers versatility but slower performance compared to established unit-testing tools.
[168]	Test Case Generation Based on Client-Server of Web Applications by Memetic Algorithm	Proposes a search-based testing approach using memetic algorithm to link client-side models with server-side code, enhancing vulnerability detection in web applications.
[166]	Atrina: Inferring Unit Oracles from GUI Test Cases	Introduces a test tool, ATRINA, that leverages DOM-based assertions and execution data to generate accurate unit-level assertions for JavaScript. The tool outperforms human-written and mutation-based methods in fault detection.
[72]	Novel approach to reuse unused test cases in a GUI based application	Addresses GUI regression testing by reusing unused test paths with varied inputs, improving test feasibility, reducing costs, and saving time for modified applications with frequent user-driven changes.
[174]	Guided Mutation Testing for JavaScript Web Applications	Introduces a mutation testing tool, MUTANDIS, for JavaScript. The tool optimizes mutation generation using static and dynamic analysis, and ranking functions by importance, significantly reducing computational costs and improving mutant relevance.
[103]	Data Flow Testing of CGI Based Web Applications	Proposes a data flow testing technique for CGI programs in Perl, modeling definition-use chains, identifying test paths for variable interactions, and enhancing the quality of web applications handling user input and database interfaces.
[179]	Search-based multi-vulnerability testing of XML injections in web applications	Presents a search-based approach, COMIX, for uncovering multiple web service vulnerabilities simultaneously. This approach improves security testing for interactions between web applications and back-end services.
[176]	SymJS: Automatic Symbolic Testing of JavaScript Web Applications	Introduces a framework (called SymJS) for automated client-side JavaScript testing that combines symbolic execution, dynamic feedback, and taint analysis to achieve high test coverage.
[172]	Automatically Generating Test Scripts for GUI Testing	Proposes a method leveraging static and dynamic analysis of application source code and executables to automate test script generation. The method achieves a substantial reduction in man-hours compared to traditional manual script creation.

Table A.18
(Continued).

Citation	Title	Study.highlights
[173]	JSEFT: Automated JavaScript Unit Test Generation	Presents a framework (called JSEFT) for generating test cases for JavaScript applications at both event and function levels. It outperforms existing test automation frameworks in achieving high coverage and fault detection accuracy.
[171]	Prioritising Server Side Reachability via Inter-process Concolic Testing	Introduces a concolic tester called StackFul for full-stack JavaScript web applications. This tool enables to distinguish high- and low-priority server errors by applying a whole-program perspective thereby improving error classification and testing efficiency.
[214]	Comparing coverage criteria for dynamic web application: An empirical evaluation	Compares the effectiveness of dynamic test coverage criteria in web applications through mutation analysis. Results show that DOM coverage is the most effective and efficient, followed by virtual DOM, HTML element, and statement coverage criteria.
[175]	Parallel evolutionary test case generation for web applications	Introduces an approach based on parallel genetic algorithm for web application test case generation. This approach improves- efficiency by reducing execution time and iterations, and effectiveness through population diversity using an island model and migration mechanism.
[177]	Mutation analysis for assessing end-to-end web tests	Presents a mutation analysis framework (called MAEWU) that evaluates web UI test suites by mutating dynamic DOM elements. Results show its effectiveness in assessing test suite adequacy and improving web app test quality.
[227]	An efficient regression testing approach for PHP web applications: a controlled experiment	Proposes a regression testing approach using impact analysis and program slicing to efficiently test modified code areas in web applications. Results show cost reduction in regression testing, depending on application characteristics and update frequency.
[281]	Using combinatorial testing to build navigation graphs for dynamic web applications	Presents a combinatorial approach (called Tansuo) for building navigation graphs in dynamic web applications. The approach generates test sequences, achieving code coverage outperforming other navigation graph tools in effectiveness.
[178]	RClassify: classifying race conditions in web applications via deterministic replay	Presents a deterministic replay-based method for identifying harmful race conditions in web applications. The method significantly reducing false positives and outperforming existing techniques in detecting real, impactful races.
[110]	Effective, Platform-Independent GUI Testing via Image Embedding and Reinforcement Learning	This study focuses on code coverage and instrumentation and suggests vision-based and reinforcement learning approaches, including platform-independent frameworks to improve the effectiveness, efficiency, and generalizability of automated GUI testing across mobile and web applications.
[180]	DBInputs: Exploiting Persistent Data to Improve Automated GUI Testing	The authors suggest that exploiting domain-specific data from application databases and improving test generation efficiency through techniques like input reuse and Q-learning can enhance automated GUI testing.
[181]	Coverage measurement in model-based testing of web applications: Tool support and an industrial experience report	This study proposes an automated model-based testing (MBT) tool, called MBTCover. The tool enables live measurement of requirements and model coverage, as well as code coverage, in large scale industrial web application testing projects.
[233]	Automation Xtreme - A Web Automation AI Tool	This study encompasses both black-box and white-box testing and introduces an AI-driven web automation tool called Xtreme-A. The tool aims at optimizing web testing through intelligent automation, enhancing efficiency, and minimizing manual effort.
[182]	Search-Based White-Box Fuzzing of Web Frontend Applications	In this study, the author emphasizes the importance of frontend testing for improved user experience and proposes a novel white-box approach using evolutionary algorithms. Existing tools are mostly black-box, limiting fault detection and no white-box tools currently exist.

Table A.19
Papers with industrial collaborations.

Title	Ref	Category
ATF - An end-to-end testing framework: experience report	[250]	Testing Framework
Cytestion: Automated GUI Testing for Web Applications	[163]	Test Generation
Automatic Web Testing Using Curiosity-Driven Reinforcement Learning	[17]	Test Generation
WebRR: self-replay enhanced robust record/replay for web application testing	[47]	Test Maintenance
TERMINATOR: better automated UI test case prioritization	[248]	Test Prioritization
Beyond Page Objects: Testing Web Applications with State Objects: Use states to drive your tests	[184]	Model Inference
Morpheus Web Testing: A Tool for Generating Test Cases for Widget Based Web Applications	[188]	Test Generation; Model Inference
Image-based Approaches for Automating GUI Testing of Interactive Web-based Applications	[190]	Oracle
Web Application Testing With Model Based Testing Method: Case Study	[191]	Testing Framework
Fostering the Diversity of Exploratory Testing in Web Applications	[126]	Test Generation; Model Inference
Automatic Generation of Tests to Exploit XML Injection Vulnerabilities in Web Applications	[236]	Test Generation
Automating Web Application Testing from the Ground Up: Experiences and Lessons Learned in an Industrial Setting	[194]	Testing Framework
Combinatorial Testing of Full Text Search in Web Applications	[258]	Test Generation
Using Semantic Similarity in Crawling-Based Web Application Testing	[195]	Test Generation; Model Inference; Oracle
Model generation to support model-based testing applied on the NASA DAT Web-application - An experience report	[197]	Test Generation; Model Inference
WebMate: Web Application Test Generation in the Real World	[200]	Test Generation; Oracle
Scripted and scriptless GUI testing for web applications: An industrial case	[134]	Test Generation
testar - scriptless testing through graphical user interface	[15]	Test Generation
Search-based multi-vulnerability testing of XML injections in web applications	[179]	Test Generation
A Search-Based Testing Approach for XML Injection Vulnerabilities in Web Applications	[237]	Test Generation
Testing concurrent user behavior of synchronous web applications with Petri nets	[116]	Test Generation; Coverage Criteria
Property-based testing of web services by deriving properties from business-rule models	[280]	Test Generation; Model Inference
An experience report on applying software testing academic results in industry: we need usable automated test generation	[238]	Test Generation
FOREPOST: finding performance problems automatically with feedback-directed learning software testing	[244]	Test Generation
A multiple case study on risk-based testing in industry	[252]	Testing Framework
GUI Testing in Production: Challenges and Opportunities	[253]	Test Generation
QExplore: An exploration strategy for dynamic web applications using guided search	[16]	Test Generation; Model Inference
Input Contract Testing of Graphical User Interfaces	[209]	Test Generation; Model Inference; Oracle
Test Automation with the Gauge Framework: Experience and Best Practices	[210]	Testing Framework
Automated Fixing of Web UI Tests via Iterative Element Matching	[57]	Test Maintenance
An automated model-based approach to repair test suites of evolving web applications	[58]	Test Maintenance
SleepReplacer: a novel tool-based approach for replacing thread sleeps in selenium WebDriver test code	[79]	Test Maintenance
Black-box model-based regression testing of fail-safe behavior in web applications	[158]	Test Generation
A case study of black box fail-safe testing in web applications	[157]	Test Generation
Reducing Flakiness in End-to-End Test Suites: An Experience Report	[80]	Test Maintenance
Automating end-to-end web testing via manual testing	[223]	Test Generation
Evaluating the TESTAR tool in an Industrial Case Study	[147]	Test Generation
Skyfire: Model-Based Testing with Cucumber	[264]	Test Generation
Towards a Robust Waiting Strategy for Web GUI Testing for an Industrial Software System	[83]	Test Maintenance
Coverage measurement in model-based testing of web applications: Tool support and an industrial experience report	[181]	Coverage Criteria
An Automatic Approach for Uniquely Discovering Actionable Elements for Systematic GUI Testing in Web Applications	[69]	Test Maintenance
Evaluation of the Choice of LLM in a Multi-Agent Solution for GUI-Test Generation	[247]	Testing Framework
LLM-Based Labelling of Recorded Automated GUI-Based Test Cases	[77]	Test Maintenance
FuzzE, Development of a Fuzzing Approach for Odoo's Tours Integration Testing Plateform	[231]	Test Generation
E2E-Loader: A Tool to Generate Performance Tests from End-to-End GUI-Level Tests	[268]	Test Generation
E2e-loader: A framework to support performance testing of web applications,	[270]	Test Generation

References

[1] F. Gandon, W. Hall, A never-ending project for humanity called "the web", in: Proceedings of the ACM Web Conference 2022, 2022, pp. 3480–3487.
[2] H.S. Xavier, The web unpacked: a quantitative analysis of global web usage, (2024). arXiv preprint arXiv:2404.17095
[3] M.T. Thielsch, I. Blotenberg, R. Jaron, User evaluation of websites: from first impression to recommendation, Interact. Comput. 26 (1) (2014) 89–102.
[4] V. Garousi, A. Mesbah, A. Betin-Can, S. Mirshokraie, A systematic mapping study of web application testing, Inf. Softw. Technol. 55 (8) (2013) 1374–1396.
[5] S. Doğan, A. Betin-Can, V. Garousi, Web application testing: a systematic literature review, J. Syst. Software 91 (2014) 174–201.
[6] A short history of the web, (https://home.cern/science/computing/birth-web/short-history-web). [Online; accessed 2-February-2025].
[7] F. Almeida, Concept and dimensions of web 4.0, Int. J. Comput. Technol. 16 (7) (2017).
[8] A. Wirfs-Brock, B. Eich, JavaScript: the first 20 years, Proc. ACM Programm. Lang. 4 (HOPL) (2020) 1–189.
[9] P. Ammann, J. Offutt, Introduction to Software Testing. Cambridge University Press, 2008.
[10] H.F. El Yamany, M.A.M. Capretz, L.F. Capretz, A multi-agent framework for testing distributed systems, in: 30th Annual International Computer Software and Applications Conference (COMPSAC'06), 2, IEEE, 2006, pp. 151–156.
[11] A. Mesbah, A. Van Deursen, D. Roest, Invariant-based automatic testing of modern web applications, IEEE Trans. Software Eng. 38 (1) (2011) 35–53.
[12] L. Mariani, M. Pezzè, O. Riganelli, M. Santoro, Automatic testing of GUI-based applications, Software Testing Verif. Reliab. 24 (5) (2014) 341–366.
[13] B.N. Nguyen, B. Robbins, I. Banerjee, A. Memon, GUITAR: an innovative tool for automated testing of GUI-driven software, Automated Software Eng. 21 (2014) 65–105.
[14] C.-Y. Wu, F. Wang, M.-H. Weng, J.-W. Lin, Automated testing of web applications with text input, in: 2015 IEEE International Conference on Progress in Informatics and Computing (PIC), IEEE, 2015, pp. 343–347.
[15] T.E.J. Vos, P. Aho, F. Pastor Ricos, O. Rodriguez-Valdes, A. Mulders, Testar–scriptless testing through graphical user interface, Software Testing Verif. Reliab. 31 (3) (2021) e1771.
[16] S. Sherin, A. Muqeet, M.U. Khan, M.Z. Iqbal, QExplore: an exploration strategy for dynamic web applications using guided search, J. Syst. Software 195 (2023) 111512.
[17] Y. Zheng, Y. Liu, X. Xie, Y. Liu, L. Ma, J. Hao, Y. Liu, Automatic web testing using curiosity-driven reinforcement learning, in: 2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE), IEEE, 2021, pp. 423–435.
[18] G.A. Di Lucca, A.R. Fasolino, Testing web-based applications: the state of the art and future trends, Inf. Softw. Technol. 48 (12) (2006) 1172–1186.
[19] P. Tonella, F. Ricca, A. Marchetto, Recent advances in web testing, Adv. Comput. 93 (2014) 1–51.
[20] Y.-F. Li, P.K. Das, D.L. Dowe, Two decades of web application testing-a survey of recent advances, Inf. Syst. 43 (2014) 20–54.
[21] P. Aho, T. Kanstren, T. Räty, J. Röning, Automated extraction of GUI models for testing, in: Advances in Computers, 95, Elsevier, 2014, pp. 49–112.
[22] A. Mesbah, Software analysis for the web: achievements and prospects, in: 2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER), 5, IEEE, 2016, pp. 91–103.
[23] M. Leotta, D. Clerissi, F. Ricca, P. Tonella, Approaches and tools for automated end-to-end web testing, in: Advances in Computers, 101, Elsevier, 2016, pp. 193–237.
[24] H. Javed, N.M. Minhas, A. Abbas, F.M. Riaz, Model based testing for web applications: a literature survey presented, J. Softw. 11 (4) (2016) 347–361.
[25] N. Gupta, V. Yadav, M. Singh, Automated regression test case generation for web application: a survey, ACM Comput. Surv. (CSUR) 51 (4) (2018) 1–25.
[26] S. Akbulut, Y.T. Gebreyesus, A. Mishra, A. Yazici, Systematic mapping on quality in web application testing, in: 2019 1st International Informatics and Software Engineering Conference (UBMYK), IEEE, 2019, pp. 1–5.
[27] F. Ricca, A. Stocco, Web test automation: insights from the grey literature, in: SOFSEM 2021: Theory and Practice of Computer Science: 47th International Conference on Current Trends in Theory and Practice of Computer Science, SOFSEM 2021, Bolzano-Bozen, Italy, January 25–29, 2021, Proceedings 47, Springer, 2021, pp. 472–485.
[28] H.A. Thooriqoh, T.N. Annisa, U.L. Yuhana, Selenium framework for web automation testing: a systematic literature review, Jurnal Ilmiah Teknologi Informasi 19 (2) (2021) 65–76.
[29] A. Alsaedi, A. Alhuzali, O. Bamasag, Black-box fuzzing approaches to secure web applications: survey, Int. J. Adv. Computer Sci. Appl. 12 (5) (2021),pp. 849–855.
[30] S. Hanna, A.A.-S. Ahmad, Web applications testing techniques: a systematic mapping study, Int. J. Web Eng. Technol. 17 (4) (2022) 372–412.
[31] N.F. Mridha, M.A. Joarder, Automated web testing over the last decade: a systematic literature review, Syst. Literat. Rev. Meta-Anal. J. 4 (1) (2023) 32–44.
[32] I. Prazina, Š. Bećirović, E. Cogo, V. Okanović, Methods for automatic web page layout testing and analysis: a review, IEEE Access 11 (2023) 13948–13964.
[33] T. Li, R. Huang, C. Cui, D. Towey, L. Ma, Y.-F. Li, W. Xia, A survey on web application testing: a decade of evolution, (2024). arXiv preprint arXiv:2412.10476
[34] S. Balsam, D. Mishra, Web application testing-challenges and opportunities, J. Syst. Software (2024) 112186.
[35] S. Mahajan, A. Alameer, P. McMinn, W.G.J. Halfond, Effective automated repair of internationalization presentation failures in web applications using style similarity clustering and search-based techniques, Software Testing Verif. Reliab. 31 (1-2) (2021) e1746.
[36] F.S. Ocariza, Jr, G. Li, K. Pattabiraman, A. Mesbah, Automatic fault localization for client-side JavaScript, Software Testing Verif. Reliab. 26 (1) (2016) 69–88.
[37] C. Wohlin, Guidelines for snowballing in systematic literature studies and a replication in software engineering, in: Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering, 2014, pp. 1–10.
[38] I. Kertusha, G. Assress, O. Duman, A. Arcuri, A survey on web testing: on the rise of AI and applications in industry, (2025). arXiv preprint arXiv:2503.05378
[39] M. Leotta, A. Stocco, F. Ricca, P. Tonella, Automated generation of visual web tests from DOM-based web tests, in: Proceedings of the 30th Annual ACM Symposium on Applied Computing, 2015, pp. 775–782.
[40] M. Leotta, B. García, F. Ricca, J. Whitehead, Challenges of end-to-end testing with selenium WebDriver and how to face them: a survey, in: 2023 IEEE Conference on Software Testing, Verification and Validation (ICST), IEEE, 2023, pp. 339–350.
[41] A. Stocco, M. Leotta, F. Ricca, P. Tonella, APOGEN: automatic page object generator for web testing. Softw. Qual. J. 25 (3) (2016) 1007–1039.
[42] Y. Liu, R. Yandrapally, A.K. Kalia, S. Sinha, R. Tzoref-Brill, A. Mesbah, CrawLabel: computing natural-language labels for UI test cases, in: Proceedings of the 3rrd ACM/IEEE International Conference on Automation of Software Test, 2022, pp. 103–114.
[43] A. Stocco, R. Yandrapally, A. Mesbah, Vista: web test repair using computer vision, in: Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, 2018, pp. 876–879.
[44] A. Stocco, R. Yandrapally, A. Mesbah, Visual web test repair, in: Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, 2018, pp. 503–514.
[45] M. Nass, E. Alégroth, R. Feldt, M. Leotta, F. Ricca, Similarity-based web element localization for robust test automation, ACM Trans. Software Eng. Method. 32 (3) (2023) 1–30.
[46] F. Shao, R. Xu, W. Haque, J. Xu, Y. Zhang, W. Yang, Y. Ye, X. Xiao, Webevo: taming web application evolution via detecting semantic structure changes, in: Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis, 2021, pp. 16–28.
[47] Z. Long, G. Wu, X. Chen, W. Chen, J. Wei, Webrr: self-replay enhanced robust record/replay for web application testing, in: Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, 2020, pp. 1498–1508.
[48] A.R. Fasolino, P. Tramontana, Towards the generation of robust E2E test cases in template-based web applications, in: 2022 48th Euromicro Conference on Software Engineering and Advanced Applications (SEAA), IEEE, 2022, pp. 104–111.
[49] H. Kirinuki, H. Tanno, K. Natsukawa, COLOR: correct locator recommender for broken test scripts using various clues in web application, in: 2019 IEEE 26th International Conference on Software Analysis, Evolution and Reengineering (SANER), IEEE, 2019, pp. 310–320.
[50] H. Pirzadeh, S. Shanian, F. Davari, A novel framework for creating user interface level tests resistant to refactoring of web applications, in: 2014 9th International Conference on the Quality of Information and Communications Technology, IEEE, 2014, pp. 268–273.

[51] Z. Khaliq, D.A. Khan, S.U. Farooq, Using deep learning for selenium web UI functional tests: a case-study with e-commerce applications, Eng. Appl. Artif. Intell. 117 (2023) 105446.
[52] M. De Luca, A.R. Fasolino, P. Tramontana, Investigating the robustness of locators in template-based web application testing using a GUI change classification model, J. Syst. Software 210 (2024) 111932.
[53] V. Nguyen, T. To, G.-H. Diep, Generating and selecting resilient and maintainable locators for web automated testing, Software Testing Verif. Reliab. 31 (3) (2021) e1760.
[54] M. Leotta, A. Stocco, F. Ricca, P. Tonella, Using multi-locators to increase the robustness of web test cases, in: 2015 IEEE 8th International Conference on Software Testing, Verification and Validation (ICST), IEEE, 2015, pp. 1–10.
[55] H.M. Eladawy, A.E. Mohamed, S.A. Salem, A new algorithm for repairing web-locators using optimization techniques, in: 2018 13th International Conference on Computer Engineering and Systems (ICCES), IEEE, 2018, pp. 327–331.
[56] M. Leotta, A. Stocco, F. Ricca, P. Tonella, ROBULA+: an algorithm for generating robust XPath locators for web testing, J. Software Evol. Process 28 (3) (2016) 177–204.
[57] Y. Lin, G. Wen, X. Gao, Automated fixing of web UI tests via iterative element matching, in: 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE), IEEE, 2023, pp. 1188–1199.
[58] J. Imtiaz, M.Z. Iqbal, et al., An automated model-based approach to repair test suites of evolving web applications, J. Syst. Software 171 (2021) 110841.
[59] W. Chen, H. Cao, X. Blanc, An improving approach for dom-based web test suite repair, in: International Conference on Web Engineering, Springer, 2021, pp. 372–387.
[60] M. Hammoudi, G. Rothermel, A. Stocco, Waterfall: an incremental approach for repairing record-replay tests of web applications, in: Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering, 2016, pp. 751–762.
[61] M. Leotta, A. Stocco, F. Ricca, P. Tonella, Pesto: automated migration of DOM-based web tests towards the visual approach, Software Testing Verif. Reliab. 28 (4) (2018) e1665.
[62] M. Leotta, F. Ricca, P. Tonella, Sidereal: statistical adaptive generation of robust locators for web testing, Software Testing Verif. Reliab. 31 (3) (2021) e1767.
[63] H. Kirinuki, S. Matsumoto, Y. Higo, S. Kusumoto, NLP-assisted web element identification toward script-free testing, in: 2021 IEEE International Conference on Software Maintenance and Evolution (ICSME), IEEE, 2021, pp. 639–643.
[64] X. Qi, X. Qian, Y. Li, Semantic test repair for web applications, in: Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, 2023, pp. 1190–1202.
[65] M. Hammoudi, Regression testing of web applications using record/replay tools, in: Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering, 2016, pp. 1079–1081.
[66] A. Stocco, How artificial intelligence can improve web development and testing, in: Companion Proceedings of the 3rd International Conference on the Art, Science, and Engineering of Programming, 2019, pp. 1–4.
[67] R. Coppola, R. Feldt, M. Nass, E. Alégroth, Ranking approaches for similarity-based web element location, J. Syst. Software 222 (2025) 112286.
[68] M. Nass, E. Alégroth, R. Feldt, Improving web element localization by using a large language model, Software Testing Verif. Reliab. 34 (7) (2024) e1893.
[69] T.S. de Moura, F.I. de Lima Mendes, E.L.G. Alves, I.R. D.S. Neto, C. de Souza Baptista, An automatic approach for uniquely discovering actionable elements for systematic GUI testing in web applications, in: 2024 IEEE 24th International Conference on Software Quality, Reliability and Security (QRS), IEEE, 2024, pp. 13–24.
[70] Z. Xu, Q. Li, S.H. Tant, Understanding and enhancing attribute prioritization in fixing web UI tests with LLMs, in: 2025 IEEE Conference on Software Testing, Verification and Validation (ICST), 2025, pp. 326–337. https://doi.org/10.1109/ICST62969.2025.10989008
[71] M. Hammoudi, G. Rothermel, P. Tonella, Why do record/replay tests of web applications break?, in: 2016 IEEE International Conference on Software Testing, Verification and Validation (ICST), IEEE, 2016, pp. 180–190.
[72] J.A. Mayan, K.L. Priya, Novel approach to reuse unused test cases in a GUI based application, in: 2015 International Conference on Circuits, Power and Computing Technologies [ICCPCT-2015], IEEE, 2015, pp. 1–7.
[73] L. Christophe, R. Stevens, C. De Roover, W. De Meuter, Prevalence and maintenance of automated functional tests for web applications, in: 2014 IEEE International Conference on Software Maintenance and Evolution, IEEE, 2014, pp. 141–150.
[74] F. Ricca, M. Leotta, A. Stocco, Three open problems in the context of e2e web testing and a vision: neonate, in: Advances in Computers, 113, Elsevier, 2019, pp. 89–133.
[75] A.M. Alakeel, Dependency detection and repair in web application tests, IAENG Int. J. Computer Sci. 49 (2) (2022).
[76] X. Gan, H. Liang, C. Brown, Challenges, strategies, and impacts: a qualitative study on UI testing in CI/CD processes from GitHub Developers' perspectives, in: 2025 IEEE Conference on Software Testing, Verification and Validation (ICST), IEEE, 2025, pp. 186–197.
[77] D.B. Franzosi, E. Alégroth, M. Isaac, LLM-based labelling of recorded automated GUI-based test cases, in: 2025 IEEE Conference on Software Testing, Verification and Validation (ICST), IEEE, 2025, pp. 453–463.
[78] S.-J. Lee, Y.-X. Chen, S.-P. Ma, W.-T. Lee, Test command auto-wait mechanisms for record and playback-style web application testing, in: 2018 IEEE 42nd Annual Computer Software and Applications Conference (COMPSAC), 2, IEEE, 2018, pp. 75–80.
[79] D. Olianas, M. Leotta, F. Ricca, SleepReplacer: a novel tool-based approach for replacing thread sleeps in selenium webdriver test code, Software Qual. J. 30 (4) (2022) 1089–1121.
[80] D. Olianas, M. Leotta, F. Ricca, L. Villa, Reducing flakiness in end-to-end test suites: an experience report, in: International Conference on the Quality of Information and Communications Technology, Springer, 2021, pp. 3–17.
[81] Y. Pei, S. Habchi, R. Rwemalika, J. Sohn, M. Papadakis, An empirical study of async wait flakiness in front-end testing, in: BENEVOL, 2022.
[82] A. Romano, Z. Song, S. Grandhi, W. Yang, W. Wang, An empirical analysis of UI-based flaky tests, in: 2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE), IEEE, 2021, pp. 1585–1597.
[83] H. Zhang, L. Liao, Z. Ding, W. Shang, N. Narula, C. Sporea, A. Toma, S. Sajedi, Towards a robust waiting strategy for web GUI testing for an industrial software system, in: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, 2024, pp. 2065–2076.
[84] Y. Pei, J. Sohn, M. Papadakis, An empirical study of web flaky tests: understanding and unveiling DOM event interaction challenges, in: 2025 IEEE Conference on Software Testing, Verification and Validation (ICST), IEEE, 2025, pp. 92–102.
[85] D. Olianas, M. Leotta, F. Ricca, Leveraging large language models for explicit wait management in end-to-end web testing, in: 2025 IEEE Conference on Software Testing, Verification and Validation (ICST), IEEE, 2025, pp. 577–581.
[86] R. Yandrapally, G. Sridhara, S. Sinha, Automated modularization of GUI test cases, in: 2015 IEEE/ACM 37th IEEE International Conference on Software Engineering, 1, IEEE, 2015, pp. 44–54.
[87] M. Leotta, M. Biagiola, F. Ricca, M. Ceccato, P. Tonella, A family of experiments to assess the impact of page object pattern in web test suite development, in: 2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST), IEEE, 2020, pp. 263–273.
[88] A. Stocco, M. Leotta, F. Ricca, P. Tonella, Clustering-aided page object generation for web testing, in: Web Engineering: 16th International Conference, ICWE 2016, Lugano, Switzerland, June 6–9, 2016. Proceedings 16, Springer, 2016, pp. 132–151.
[89] A. Stocco, M. Leotta, F. Ricca, P. Tonella, Automatic page object generation with APOGEN, in: Web Engineering: 16th International Conference, ICWE 2016, Lugano, Switzerland, June 6–9, 2016. Proceedings 16, Springer, 2016, pp. 533–537.
[90] M. Leotta, A. Molinari, F. Ricca, Assessor: a po-based webdriver test suites generator from selenium ide recordings, in: 2022 IEEE Conference on Software Testing, Verification and Validation (ICST), IEEE, 2022, pp. 389–399.
[91] A. Stocco, M. Leotta, F. Ricca, P. Tonella, Why creating web page objects manually if it can be done automatically?, in: 2015 IEEE/ACM 10th International Workshop on Automation of Software Test, IEEE, 2015, pp. 70–74.
[92] M.M.N. Fuad, K. Sakib, WebEV: a dataset on the behavior of testers for web application end to end testing, in: 2023 IEEE/ACM 31st International Conference on Program Comprehension (ICPC), IEEE, 2023, pp. 79–83.

[93] S. Di Meglio, End-to-end testing in web environments: addressing practical challenges, in: 2025 IEEE Conference on Software Testing, Verification and Validation (ICST), IEEE, 2025, pp. 782–784.
[94] O. Parry, G.M. Kapfhammer, M. Hilton, P. McMinn, A survey of flaky tests, ACM Trans. Software Eng. Methodol. (TOSEM) 31 (1) (2021) 1–74.
[95] M.M. Kamal, S.M. Darwish, A. Elfatatry, Enhancing the automation of GUI testing, in: Proceedings of the 8th International Conference on Software and Information Engineering, 2019, pp. 66–70.
[96] M.M. Öztürk, A. Zengin, Improved GUI testing using task parallel library, ACM SIGSOFT Software Eng. Notes 41 (2) (2016) 1–8.
[97] D. Zimmermann, A. Koziolek, GUI-based software testing: an automated approach using GPT-4 and selenium WebDriver, in: 2023 38th IEEE/ACM International Conference on Automated Software Engineering Workshops (ASEW), IEEE, 2023, pp. 171–174.
[98] N. Pombo, C. Martins, Test driven development in action: case study of a cross-platform web application, in: IEEE EUROCON 2021-19th International Conference on Smart Technologies, IEEE, 2021, pp. 352–356.
[99] S. Wang, W. Wu, J. Sun, A method for test cases reduction in web application testing based on user session, in: 2018 International Conference on Networking and Network Applications (NaNA), IEEE, 2018, pp. 378–383.
[100] H. Lu, L. Wang, M. Ye, K. Yan, Q. Jin, DNN-based image classification for software gui testing, in: 2018 IEEE SmartWorld, Ubiquitous Intelligence & Computing, Advanced & Trusted Computing, Scalable Computing & Communications, Cloud & Big Data Computing, Internet of People and Smart City Innovation (SmartWorld/SCALCOM/UIC/ATC/CBDCom/IOP/SCI), IEEE, 2018, pp. 1818–1823.
[101] S. Kiran, A. Mohapatra, R. Swamy, Experiences in performance testing of web applications with unified authentication platform using jmeter, in: 2015 International Symposium on Technology Management and Emerging Technologies (ISTMET), IEEE, 2015, pp. 74–78.
[102] H. Tanno, X. Zhang, Test script generation based on design documents for web application testing, in: 2015 IEEE 39th Annual Computer Software and Applications Conference, 3, IEEE, 2015, pp. 672–673.
[103] M. Sahu, D.P. Mohapatra, Data flow testing of CGI based web applications, in: 2014 International Conference on Information Technology, IEEE, 2014, pp. 106–111.
[104] X. Li, H. Zeng, Modeling web application for cross-browser compatibility testing, in: 15th IEEE/ACIS International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD), IEEE, 2014, pp. 1–5.
[105] J.P. Sotomayor, S.C. Allala, D. Santiago, T.M. King, P.J. Clarke, Comparison of open-source runtime testing tools for microservices Softw. Qual. J. 31 (a3) (2022) 55–87.
[106] K. Manukonda, A comprehensive evaluation of selenium webdriver for cross-browser test automation: performance, reliability, and usability, J. Artif. Intell. Mach. Learn. Data Sci. 1 (3) (2023) 580–584.
[107] T. Tanaka, H. Niibori, L. Shiyingxue, S. Nomura, T. Nakao, K. Tsuda, Selenium based testing systems for analytical data generation of website user behavior, in: 2020 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW), IEEE, 2020, pp. 216–221.
[108] A. Andrews, S. Boukhris, S. Elakeili, Fail-safe testing of web applications, in: 2014 23rd Australian Software Engineering Conference, 2014, pp. 200–209. https://doi.org/10.1109/ASWEC.2014.29
[109] R. Aekaukkharachawakit, T. Suwannasart, Generating a robot framework test script for web application based on database constraints, in: Proceeding of the 2024 5th Asia Service Sciences and Software Engineering Conference, 2024, pp. 1–12.
[110] S. Yu, C. Fang, X. Li, Y. Ling, Z. Chen, Z. Su, Effective, platform-independent GUI testing via image embedding and reinforcement learning, ACM Trans. Software Eng. Method. 33 (7) (2024) 1–27.
[111] L.V. Hufkens, F. Pastor Ricos, B. Marín, T.E.J. Vos, Grammar-based action selection rules for scriptless testing, in: Proceedings of the 5th ACM/IEEE International Conference on Automation of Software Test (AST 2024), 2024, pp. 56–65.
[112] H. Huynh, N. Pham, T.N. Nguyen, V. Nguyen, Segment-based test case prioritization: a multi-objective approach, in: Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, 2024, pp. 1149–1160.
[113] Y. Huang, M. Lu, X. Long, Semantic-guided adaptive exploration testing for GUI software, in: 2024 11th International Conference on Dependable Systems and Their Applications (DSA), IEEE, 2024, pp. 141–149.
[114] T. Li, C. Cui, R. Huang, D. Towey, L. Ma, Large language models for automated web-form-test generation: an empirical study, ACM Trans. Software Eng. Method. 3,pp. 1–37.
[115] C. Augusto, J. Morán, A. Bertolino, C. de la Riva, J. Tuya, Software system testing assisted by large language models: an exploratory study, in: IFIP International Conference on Testing Software and Systems, Springer, 2024, pp. 239–255.
[116] J. Offutt, S. Thummala, Testing concurrent user behavior of synchronous web applications with petri nets, Software Syst. Model. 18 (2019) 913–936.
[117] S.B. Nettur, S. Karpurapu, U. Nettur, L.S. Gajja, Cypress copilot: development of an AI assistant for boosting productivity and transforming web application testing, IEEE Access (2024).
[118] D.I.M. SERGIO, L.L.L. Starace, V. Pontillo, R. Opdebeeck, C. De Roover, S. Di Martino, Investigating the adoption and maintenance of web gui testing: insights from github repositories, Available at SSRN 5182165 (2025).
[119] A. Corazza, S. Di Martino, A. Peron, L.L.L. Starace, Web application testing: using tree kernels to detect near-duplicate states in automated model inference, in: Proceedings of the 15th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM), 2021, pp. 1–6.
[120] O. Loubiri, S. Maag, Automated web testing using machine learning and containerization, in: 2022 26th International Conference on Circuits, Systems, Communications and Computers (CSCC), IEEE, 2022, pp. 113–121.
[121] D.P. Nguyen, S. Maag, Codeless web testing using selenium and machine learning, in: ICSOFT 2020: 15th International Conference on Software Technologies, ScitePress, 2020, pp. 51–60.
[122] V. Garousi, D. Pfahl, J.M. Fernandes, M. Felderer, M.V. Mäntylä, D. Shepherd, A. Arcuri, A. Coşkunçay, B. Tekinerdogan, Characterizing industry-academia collaborations in software engineering: evidence from 101 projects, Empir. Software Eng. 24 (4) (2019) 2540–2602.
[123] F. Azzam, A. Saies, M. Jaber, S. Zein, Evaluation for web GUI automation testing tool-experiment, in: 2022 International Symposium on Multidisciplinary Studies and Innovative Technologies (ISMSIT), IEEE, 2022, pp. 549–554.
[124] R.M. Moreira, A.C. Paiva, M. Nabuco, A. Memon, Pattern-based GUI testing: bridging the gap between design and quality assurance, Software Testing Verif. Reliab. 27 (3) (2017) e1629.
[125] T. Fulcini, L. Ardito, Gamified exploratory GUI testing of web applications: a preliminary evaluation, in: 2022 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW), IEEE, 2022, pp. 215–222.
[126] J. Leveau, X. Blanc, L. Réveillère, J.-R. Falleri, R. Rouvoy, Fostering the diversity of exploratory testing in web applications, Software Testing Verif. Reliab. 32 (5) (2022) e1827.
[127] R. Cassino, M. Tucci, G. Vitiello, R. Francese, Empirical validation of an automatic usability evaluation method, J. Visual Lang. Comput. 28 (2015) 1–22.
[128] S. Thummala, J. Offutt, Using Petri nets to test concurrent behavior of web applications, in: 2016 IEEE Ninth International Conference on Software Testing, Verification and Validation Workshops (ICSTW), IEEE, 2016, pp. 189–198.
[129] S. Firmenich, A. Garrido, J. Grigera, J.M. Rivero, G. Rossi, Usability improvement through A/B testing and refactoring, Software Qual. J. 27 (2019) 203–240.
[130] M. Leotta, F. Ricca, A. Marchetto, D. Olianas, An empirical study to compare three web test automation approaches: NLP-based, programmable, and capture&replay, J. Software Evol. Process 36 (5) (2024) e2606.
[131] M. Leotta, D. Paparella, F. Ricca, Comparing the effectiveness of assertions with differential testing in the context of web testing, in: International Conference on the Quality of Information and Communications Technology, Springer, 2022, pp. 108–124.
[132] F. Ricca, M. Leotta, Towards automated generation of PO-based WebDriver test suites from selenium IDE recordings, in: Proceedings of the 12th International Workshop on Automating TEST Case Design, Selection, and Evaluation, 2021, pp. 9–16.
[133] S. Hanna, H. Jaber, An approach for web applications test data generation based on analyzing client side user input fields, in: 2019 2nd International Conference on New Trends in Computing Sciences (ICTCS), IEEE, 2019, pp. 1–6.
[134] A. Bons, B. Marín, P. Aho, T.E.J. Vos, Scripted and scriptless GUI testing for web applications: an industrial case, Inf. Softw. Technol. 158 (2023) 107172.

[135] N. Sunman, Y. Soydan, H. Sözer, Automated web application testing driven by pre-recorded test cases, J. Syst. Software 193 (2022) 111441.
[136] Y.-H. Long, Y.-C. Chen, X.-P. Chen, X.-H. Shi, F. Zhou, Test-driven feature extraction of web components, J. Comput. Sci. Technol. 37 (2) (2022) 389–404.
[137] G.R. Mattiello, A.T. Endo, Model-based testing leveraged for automated web tests, Software Qual. J. 30 (3) (2022) 621–649.
[138] J. Eskonen, J. Kahles, J. Reijonen, Automating gui testing with image-based deep reinforcement learning, in: 2020 IEEE International Conference on Autonomic Computing and Self-Organizing Systems (ACSOS), IEEE, 2020, pp. 160–167.
[139] R. Coppola, T. Fulcini, L. Ardito, M. Torchiano, E. Alègroth, On effectiveness and efficiency of gamified exploratory GUI testing, IEEE Trans. Software Eng. (2023).
[140] R. Coppola, T. Fulcini, M. Torchiano, Conceptualization of multi-user collaborative GUI-testing for web applications, in: International Conference on the Quality of Information and Communications Technology, Springer, 2024, pp. 110–125.
[141] G. Garaccione, T. Fulcini, P. Stefanut Bodnarescul, R. Coppola, L. Ardito, Gamified GUI testing with selenium in the IntelliJ IDE: a prototype plugin, in: Proceedings of the 1st ACM/IEEE Workshop on Integrated Development Environments, 2024, pp. 76–80.
[142] V. Nguyen, B. Le, RLTCP: A reinforcement learning approach to prioritizing automated user interface tests, Inf. Softw. Technol. 136 (2021) 106574.
[143] B. García, M. Gallego, F. Gortázar, M. Munoz-Organero, A survey of the selenium ecosystem, Electronics 9 (7) (2020) 1067.
[144] B. García, M. Munoz-Organero, C. Alario-Hoyos, C.D. Kloos, Automated driver management for selenium WebDriver, Empir. Software Eng. 26 (2021) 1–51.
[145] M. Leotta, B. García, F. Ricca, An empirical study to quantify the setup and maintenance benefits of adopting WebDriverManager, in: International Conference on the Quality of Information and Communications Technology, Springer, 2022, pp. 31–45.
[146] R. Othman, S. Zein, Test case auto-generation for web applications: a model-based approach, in: 2022 International Symposium on Multidisciplinary Studies and Innovative Technologies (ISMSIT), IEEE, 2022, pp. 18–25.
[147] S. Bauersfeld, T.E.J. Vos, N. Condori-Fernández, A. Bagnato, E. Brosse, Evaluating the TESTAR tool in an industrial case study, in: Proceedings of the 8th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement, 2014, pp. 1–9.
[148] S. Wang, S. Wang, Y. Fan, X. Li, Y. Liu, Leveraging large vision-language model for better automatic web GUI testing, in: 2024 IEEE International Conference on Software Maintenance and Evolution (ICSME), IEEE, 2024, pp. 125–137.
[149] J. Guo, B. Yu, L. Liu, W. Wang, Multi-objective test case generation for web applications with limited resources, in: 2024 IEEE 48th Annual Computers, Software, and Applications Conference (COMPSAC), IEEE, 2024, pp. 1310–1315.
[150] M. Bures, Framework for assessment of web application automated testability, in: Proceedings of the 2015 Conference on Research in Adaptive and Convergent Systems, 2015, pp. 512–514.
[151] A.I. Esparcia-Alcázar, F. Almenar, M. Martínez, U. Rueda, T. Vos, Q-learning strategies for action selection in the TESTAR automated testing tool, 6th International Conferenrence on Metaheuristics and nature inspired computing (META 2016) (2016) 130–137.
[152] A.I. Esparcia-Alcázar, F. Almenar, T.E.J. Vos, U. Rueda, Using genetic programming to evolve action selection rules in traversal-based automated software testing: results obtained with the TESTAR tool, Memetic Comput. 10 (2018) 257–265.
[153] F.P. Ricós, A. Slomp, B. Marín, P. Aho, T.E.J. Vos, Distributed state model inference for scriptless GUI testing, J. Syst. Software 200 (2023) 111645.
[154] O. Rodriguez-Valdes, Towards a testing tool that learns to test, in: 2021 IEEE/ACM 43rd International Conference on Software Engineering: Companion Proceedings (ICSE-Companion), IEEE, 2021, pp. 278–280.
[155] F. Almenar, A.I. Esparcia-Alcázar, M. Martínez, U. Rueda, Automated testing of web applications with TESTAR: lessons learned testing the odoo tool, in: Search Based Software Engineering: 8th International Symposium, SSBSE 2016, Raleigh, NC, USA, October 8–10, 2016, Proceedings 8, Springer, 2016, pp. 218–223.
[156] U. Rueda, T.E.J. Vos, F. Almenar, M.O. Martınez, A.I. Esparcia-Alcázar, TESTAR: from academic prototype towards an industry-ready tool for automated testing at the user interface level, Actas de las XX Jornadas de Ingenierıa del Software y Bases de Datos (JISBD 2015) (2015) 236–245.
[157] S. Boukhris, A. Andrews, A. Alhaddad, R. Dewri, A case study of black box fail-safe testing in web applications, J. Syst. Software 131 (2017) 146–167.
[158] A. Andrews, A. Alhaddad, S. Boukhris, Black-box model-based regression testing of fail-safe behavior in web applications, J. Syst. Software 149 (2019) 318–339.
[159] M. Biagiola, A. Stocco, F. Ricca, P. Tonella, Diversity-based web test generation, in: Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, 2019, pp. 142–153.
[160] M. Pinto, M. Gonçalves, P. Masci, J.C. Campos, TOM: a model-based gui testing framework, in: Formal Aspects of Component Software: 14th International Conference, FACS 2017, Braga, Portugal, October 10–13, 2017, Proceedings 14, Springer, 2017, pp. 155–161.
[161] B. Yu, L. Ma, C. Zhang, Incremental web application testing using page object, in: 2015 Third IEEE Workshop on Hot Topics in Web Systems and Technologies (HotWeb), IEEE, 2015, pp. 1–6.
[162] G. Chen, G. Chen, D. Wu, Q. Liu, L. Zhang, X. Fan, A selenium-based web application automation test framework, in: 2021 IEEE 2nd International Conference on Information Technology, Big Data and Artificial Intelligence (ICIBA), 2, IEEE, 2021, pp. 257–261.
[163] T.S.d. Moura, E.L.G. Alves, H.F.d. Figueirêdo, B.C. de Souza, Cytestion: automated GUI testing for web applications, in: Proceedings of the XXXVII Brazilian Symposium on Software Engineering, 2023, pp. 388–397.
[164] M.M.N. Fuad, K. Sakib, Ebat: an efficient automated web application testing approach based on tester's behavior, in: 2022 29th Asia-Pacific Software Engineering Conference (APSEC), IEEE, 2022, pp. 482–486.
[165] Y. Fan, S. Wang, Z. Fei, Y. Qin, H. Li, Y. Liu, Can cooperative multi-agent reinforcement learning boost automatic web testing? An exploratory study, in: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, 2024, pp. 14–26.
[166] S. Mirshokraie, A. Mesbah, K. Pattabiraman, Atrina: inferring unit oracles from GUI test cases, in: 2016 IEEE International Conference on Software Testing, Verification and Validation (ICST), IEEE, 2016, pp. 330–340.
[167] X. Chang, Z. Liang, Y. Zhang, L. Cui, Z. Long, G. Wu, Y. Gao, W. Chen, J. Wei, T. Huang, A reinforcement learning approach to generating test cases for web applications, in: 2023 IEEE/ACM International Conference on Automation of Software Test (AST), IEEE, 2023, pp. 13–23.
[168] W. Wang, X. Guo, Z. Li, R. Zhao, Test case generation based on client-server of web applications by memetic algorithm, in: 2019 IEEE 30th International Symposium on Software Reliability Engineering (ISSRE), IEEE, 2019, pp. 206–216.
[169] M. Hirzel, H. Klaeren, Code coverage for any kind of test in any kind of transcompiled cross-platform applications, in: Proceedings of the 2nd International Workshop on User Interface Test Automation, 2016, pp. 1–10.
[170] D.T.H. Thu, D.-A. Nguyen, P.N. Hung, Automated test data generation for typescript web applications, in: 2021 13th International Conference on Knowledge and Systems Engineering (KSE), IEEE, 2021, pp. 1–6.
[171] M. Vandercammen, L. Christophe, D. Di Nucci, W. De Meuter, C. De Roover, et al., Prioritising server side reachability via inter-process concolic testing, Art Sci. Eng. Programm. 5 (2) (2021) 1–38.
[172] M. Iyama, H. Kirinuki, H. Tanno, T. Kurabayashi, Automatically generating test scripts for gui testing, in: 2018 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW), IEEE, 2018, pp. 146–150.
[173] S. Mirshokraie, A. Mesbah, K. Pattabiraman, Jseft: automated javascript unit test generation, in: 2015 IEEE 8th International Conference on Software Testing, Verification and Validation (ICST), IEEE, 2015, pp. 1–10.
[174] S. Mirshokraie, A. Mesbah, K. Pattabiraman, Guided mutation testing for javascript web applications, IEEE Trans. Software Eng. 41 (5) (2014) 429–444.
[175] W. Wang, S. Wu, Z. Li, R. Zhao, Parallel evolutionary test case generation for web applications, Inf. Softw. Technol. 155 (2023) 107113.
[176] G. Li, E. Andreasen, I. Ghosh, SymJS: automatic symbolic testing of JavaScript web applications, in: Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering, 2014, pp. 449–459.
[177] R. Yandrapally, A. Mesbah, Mutation analysis for assessing end-to-end web tests, in: 2021 IEEE International Conference on Software Maintenance and Evolution (ICSME), IEEE, 2021, pp. 183–194.
[178] L. Zhang, C. Wang, RClassify: classifying race conditions in web applications via deterministic replay, in: 2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE), IEEE, 2017, pp. 278–288.
[179] S. Jan, A. Panichella, A. Arcuri, L. Briand, Search-based multi-vulnerability testing of XML injections in web applications, Empir. Software Eng. 24 (6) (2019) 3696–3729.

[180] D. Clerissi, G. Denaro, M. Mobilio, L. Mariani, DBINPUTS: exploiting persistent data to improve automated GUI testing, IEEE Trans. Software Eng. (2024).
[181] V. Garousi, A.B. Keleş, Y. Balaman, A. Mermer, Z.Ö. Güler, Coverage measurement in model-based testing of web applications: tool support and an industrial experience report, in: 2024 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW), IEEE, 2024, pp. 37–43.
[182] I. Kertusha, Search-based white-box fuzzing of web frontend applications, in: 2024 IEEE 35th International Symposium on Software Reliability Engineering Workshops (ISSREW), IEEE, 2024, pp. 95–98.
[183] A. Rau, J. Hotzkow, A. Zeller, Efficient GUI test generation by learning from tests of other apps, in: Proceedings of the 40th International Conference on Software Engineering: Companion Proceeedings, 2018, pp. 370–371.
[184] A. Van Deursen, Beyond page objects: testing web applications with state objects: use states to drive your tests, Queue 13 (6) (2015) 20–37.
[185] R. Yandrapally, S. Sinha, R. Tzoref-Brill, A. Mesbah, Carving UI tests to generate api tests and api specification, in: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), IEEE, 2023, pp. 1971–1982.
[186] Y. Fan, S. Wang, S. Wang, Y. Liu, G. Wen, Q. Rong, A comprehensive evaluation of Q-learning based automatic web GUI testing, in: 2023 10th International Conference on Dependable Systems and Their Applications (DSA), IEEE, 2023, pp. 12–23.
[187] K. Waitchasarn, A. Thongtak, T. Suwannasart, Generating robot framework test scripts from user stories and scenarios for web application testing, in: 2023 8th International Conference on Computer and Communication Systems (ICCCS), IEEE, 2023, pp. 795–801.
[188] R. de Almeida Neves, W.M. Watanabe, R. Oliveira, Morpheus web testing: a tool for generating test cases for widget based web applications, J. Web Eng. 21 (2) (2022) 119–144.
[189] K. Jaganeshwari, S. Djodilatchoumy, A novel approach of gui mapping with image based widget detection and classification, in: 2021 2nd International Conference on Intelligent Engineering and Management (ICIEM), IEEE, 2021, pp. 342–346.
[190] F. Macchi, P. Rosin, J.M. Mervi, L. Turchet, Image-based approaches for automating GUI testing of interactive web-based applications, in: 2021 28th Conference of Open Innovations Association (FRUCT), IEEE, 2021, pp. 278–285.
[191] P. Akpinar, M.S. Aktas, A.B. Keles, Y. Balaman, Z.O. Guler, O. Kalipsiz, Web application testing with model based testing method: case study, in: 2020 International Conference on Electrical, Communication, and Computer Engineering (ICECCE), IEEE, 2020, pp. 1–6.
[192] V. Nayak, D. Kraus, Session-based recommender systems for action selection in gui test generation, in: 2020 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW), IEEE, 2020, pp. 372–375.
[193] H. Tanaka, X-brot: prototyping of compatibility testing tool for web application based on document analysis technology, in: 2019 International Conference on Document Analysis and Recognition Workshops (ICDARW), 7, IEEE, 2019, pp. 18–21.
[194] V. Debroy, L. Brimble, M. Yost, A. Erry, Automating web application testing from the ground up: experiences and lessons learned in an industrial setting, in: 2018 IEEE 11th International Conference on Software Testing, Verification and Validation (ICST), IEEE, 2018, pp. 354–362.
[195] J.-W. Lin, F. Wang, P. Chu, Using semantic similarity in crawling-based web application testing, in: 2017 IEEE International Conference on Software Testing, Verification and Validation (ICST), IEEE, 2017, pp. 138–148.
[196] X. Dawei, J. Liqiu, X. Xinpeng, W. Yuhang, Web application automatic testing solution, in: 2016 3rd International Conference on Information Science and Control Engineering (ICISCE), IEEE, 2016, pp. 1183–1187.
[197] C. Schulze, M. Lindvall, S. Bjorgvinsson, R. Wiegand, Model generation to support model-based testing applied on the NASA DAT web-application-An experience report, in: 2015 IEEE 26th International Symposium on Software Reliability Engineering (ISSRE), IEEE, 2015, pp. 77–87.
[198] G. Brajnik, A. Baruzzo, S. Fabbro, Model-based continuous integration testing of responsiveness of web applications, in: 2015 IEEE 8th International Conference on Software Testing, Verification and Validation (ICST), IEEE, 2015, pp. 1–2.
[199] K.V. Arya, H. Verma, Keyword driven automated testing framework for web application, in: 2014 9th International Conference on Industrial and Information Systems (ICIIS), IEEE, 2014, pp. 1–6.
[200] V. Dallmeier, B. Pohl, M. Burger, M. Mirold, A. Zeller, WebMate: web application test generation in the real world, in: 2014 IEEE Seventh International Conference on Software Testing, Verification and Validation Workshops, IEEE, 2014, pp. 413–418.
[201] R.F. Oguz, I. Erdem, E. Olmezogullari, M.S. Aktas, End-to-end automated UI testing workflow for web sites with intensive user–system interactions, Int. J. Software Eng. Knowl. Eng. 32 (10) (2022) 1477–1497.
[202] I. Erdem, R.F. Oguz, E. Olmezogullari, M.S. Aktas, Test script generation based on hidden Markov models learning from user browsing behaviors, in: 2021 IEEE International Conference on Big Data (Big Data), IEEE, 2021, pp. 2998–3005.
[203] A.C.R. Paiva, A. Restivo, S. Almeida, Test case generation based on mutations over user execution traces, Software Qual. J. 28 (3) (2020) 1173–1186.
[204] A. Halin, A. Nuttinck, M. Acher, X. Devroey, G. Perrouin, B. Baudry, Test them all, is it worth it? Assessing configuration sampling on the JHipster web development stack, Empir. Software Eng. 24 (2019) 674–717.
[205] M. Biagiola, F. Ricca, P. Tonella, Search based path and input data generation for web application testing, in: Search Based Software Engineering: 9th International Symposium, SSBSE 2017, Paderborn, Germany, September 9–11, 2017, Proceedings 9, Springer, 2017, pp. 18–32.
[206] T.A. Walsh, P. McMinn, G.M. Kapfhammer, Automatic detection of potential layout faults following changes to responsive web pages (N), in: 2015 30th IEEE/ACM International Conference on Automated Software Engineering (ASE), IEEE, 2015, pp. 709–714.
[207] E. Vila, G. Novakova, D. Todorova, Automation testing framework for web applications with selenium WebDriver: opportunities and threats, in: Proceedings of the International Conference on Advances in Image Processing, 2017, pp. 144–150.
[208] D. Clerissi, M. Leotta, G. Reggio, F. Ricca, Towards the generation of end-to-end web test scripts from requirements specifications, in: 2017 IEEE 25th International Requirements Engineering Conference Workshops (REW), IEEE, 2017, pp. 343–350.
[209] T. Tuglular, F. Belli, M. Linschulte, Input contract testing of graphical user interfaces, Int. J. Software Eng. Knowl. Eng. 26 (02) (2016) 183–215.
[210] V. Garousi, A.B. Keleş, Y. Balaman, Z.Ö. Güler, Test automation with the gauge framework: experience and best practices, in: Computational Science and Its Applications–ICCSA 2020: 20th International Conference, Cagliari, Italy, July 1–4, 2020, Proceedings, Part II 20, Springer, 2020, pp. 458–470.
[211] H. Tanaka, Automating web GUI compatibility testing using X-BROT: prototyping and field trial, in: Document Analysis and Recognition–ICDAR 2021 Workshops: Lausanne, Switzerland, September 5–10, 2021, Proceedings, Part II 16, Springer, 2021, pp. 255–267.
[212] A. Milani Fard, M. Mirzaaghaei, A. Mesbah, Leveraging existing tests in automated test generation for web applications, in: Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering, 2014, pp. 67–78.
[213] S.M. Shariff, H. Li, C.-P. Bezemer, A.E. Hassan, T.H.D. Nguyen, P. Flora, Improving the testing efficiency of selenium-based load tests, in: 2019 IEEE/ACM 14th International Workshop on Automation of Software Test (AST), IEEE, 2019, pp. 14–20.
[214] S. Sherin, M.Z. Iqbal, M.U. Khan, A.A. Jilani, Comparing coverage criteria for dynamic web application: an empirical evaluation, Computer Stand. Interfaces 73 (2021) 103467.
[215] M. Biagiola, A. Stocco, A. Mesbah, F. Ricca, P. Tonella, Web test dependency detection, in: Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, 2019, pp. 154–164.
[216] D. Olianas, M. Leotta, F. Ricca, M. Biagiola, P. Tonella, Stile: a tool for parallel execution of e2e web test scripts, in: 2021 14th IEEE Conference on Software Testing, Verification and Validation (ICST), IEEE, 2021, pp. 460–465.
[217] D. Clerissi, M. Leotta, G. Reggio, F. Ricca, Test driven development of web applications: a lightweight approach, in: 2016 10th International Conference on the Quality of Information and Communications Technology (QUATIC), IEEE, 2016, pp. 25–34.
[218] B. García, C.D. Kloos, C. Alario-Hoyos, M. Munoz-Organero, Selenium-jupiter: a junit 5 extension for selenium webdriver, J. Syst. Software 189 (2022) 111298.
[219] M. Leotta, F. Ricca, S. Stoppa, A. Marchetto, Is NLP-based test automation cheaper than programmable and capture &replay?, in: International Conference on the Quality of Information and Communications Technology, Springer, 2022, pp. 77–92.
[220] S. Fatima, Comparative study of software automation tools: selenium and quick test professional, Journal of Independent Studies and Research Computing 21 (1) (2023).
[221] Y. Chicas, S. Maag, An automated parallel compatibility testing framework fo web-based systems, in: ICWI AC 2021: 20th International Conference on WWW/Internet and 18th International Conference on Applied Computing, International Assn for Development of the Information Society (IADIS), 2021, pp. 163–174.

[222] R. Khankhoje, Web page element identification using selenium and cnn: a novel approach, J. Software Qual. Assur. 1 (1) (2023).
[223] H. Kirinuki, H. Tanno, Automating end-to-end web testing via manual testing, J. Inf. Process. 30 (2022) 294–306.
[224] A.C. Barus, A. Sembiring, Website functionality testing using the automation tool, in: 2023 IEEE International Conference on Data and Software Engineering (ICoDSE), IEEE, 2023, pp. 256–260.
[225] M. Dadkhah, S. Araban, S. Paydar, A semantic web enabled approach to automate test script generation for web applications, Computer Knowl. Eng. 4 (1) (2021) 11–24.
[226] H.P. Nguyen, H.A. Le, N.T. Truong, JFAT: an automation framework for web application testing, in: Context-Aware Systems and Applications, and Nature of Computation and Communication: 7th EAI International Conference, ICCASA 2018, and 4th EAI International Conference, ICTCC 2018, Viet Tri City, Vietnam, November 22–23, 2018, Proceedings 7, Springer, 2019, pp. 48–57.
[227] H. Do, M. Hossain, An efficient regression testing approach for PHP web applications: a controlled experiment, Software Testing Verif. Reliab. 24 (5) (2014) 367–385.
[228] P.T. Chiou, A.S. Alotaibi, W.G.J. Halfond, Detecting dialog-related keyboard navigation failures in web applications, in: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), IEEE, 2023, pp. 1368–1380.
[229] I. Verma, R. Wason, P. Arora, TEST-web: an automated web application testing tool for enhanced efficiency and consistency, in: 2024 3rd Edition of IEEE Delhi Section Flagship Conference (DELCON), IEEE, 2024, pp. 1–4.
[230] P. Sriwichainan, T. Suwannasart, Generating test scripts for web-based applications, in: Proceedings of the 2020 3rd International Conference on Sensors, Signal and Image Processing, 2020, pp. 84–89.
[231] G. Benoit, F. Georis, G. Debongnie, B. Vanderose, X. Devroey, FuzzE, development of a fuzzing approach for Odoo's tours integration testing plateform, in: 2025 IEEE Conference on Software Testing, Verification and Validation (ICST), IEEE, 2025, pp. 533–541.
[232] B. Hnatkowska, K. Wójcikowski, Functional test generation based on requirements defined at GUI level, in: Asian Conference on Intelligent Information and Database Systems, Springer, 2024, pp. 326–336.
[233] R. Abishek, M. Vijay Anand, Automation xtreme-A web automation AI tool, in: International Conference on Computational Intelligence in Data Science, Springer, 2024, pp. 51–64.
[234] P. Jiang, S. Wang, Y. Liu, Tree-based synthesis of web test sequences from manual actions, in: International Symposium on Theoretical Aspects of Software Engineering, Springer, 2024, pp. 242–260.
[235] I.V. Krasnokutska, O.S. Krasnokutskyi, Implementing E2E tests with cypress and page object model: evolution of approaches, in: CS&SE@ SW, 2023, pp. 101–110.
[236] S. Jan, A. Panichella, A. Arcuri, L. Briand, Automatic generation of tests to exploit XML injection vulnerabilities in web applications, IEEE Trans. Software Eng. 45 (4) (2017) 335–362.
[237] S. Jan, C.D. Nguyen, A. Arcuri, L. Briand, A search-based testing approach for XML injection vulnerabilities in web applications, in: 2017 IEEE International Conference on Software Testing, Verification and Validation (ICST), IEEE, 2017, pp. 356–366.
[238] A. Arcuri, An experience report on applying software testing academic results in industry: we need usable automated test generation, Empir. Software Eng. 23 (2018) 1959–1981.
[239] N. Jayalakshmi, K. Sakthivel, A hybrid approach for automated GUI testing using quasi-oppositional genetic sparrow search algorithm, in: 2024 International Conference on Innovative Computing, Intelligent Communication and Smart Electrical Systems (ICSES), IEEE, 2024, pp. 1–7.
[240] M. Mobilio, D. Clerissi, G. Denaro, L. Mariani, GUI testing to the power of parallel Q-Learning, in: 2023 IEEE/ACM 45th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion), IEEE, 2023, pp. 55–59.
[241] K. Ivanova, G.V. Kondratenko, I.V. Sidenko, Y.P. Kondratenko, Artificial intelligence in automated system for web-interfaces visual testing, in: COLINS, 2020, pp. 1019–1031.
[242] R. Yandrapally, A. Stocco, A. Mesbah, Near-duplicate detection in web app model inference, in: Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering, 2020, pp. 186–197.
[243] J.-H. Kwon, I.-Y. Ko, G. Rothermel, Prioritizing browser environments for web application test execution, in: Proceedings of the 40th International Conference on Software Engineering, 2018, pp. 468–479.
[244] Q. Luo, A. Nair, M. Grechanik, D. Poshyvanyk, Forepost: finding performance problems automatically with feedback-directed learning software testing, Empir. Software Eng. 22 (2017) 6–56.
[245] M. Ermuth, M. Pradel, Monkey see, monkey do: effective generation of GUI tests with inferred macro events, in: Proceedings of the 25th International Symposium on Software Testing and Analysis, 2016, pp. 82–93.
[246] C. Liu, J. Wang, W. Yang, Y. Zhang, T. Xie, Judge: effective State Abstraction for Guiding Automated Web GUI Testing, ACM Trans. Software Eng. Method. 35, pp. 1–40.
[247] S. Tomic, E. Alégroth, M. Isaac, Evaluation of the choice of LLM in a multi-agent solution for GUI-test generation, in: 2025 IEEE Conference on Software Testing, Verification and Validation (ICST), IEEE, 2025, pp. 487–497.
[248] Z. Yu, F. Fahid, T. Menzies, G. Rothermel, K. Patrick, S. Cherian, Terminator: better automated ui test case prioritization, in: Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, 2019, pp. 883–894.
[249] S. Bauersfeld, T.E.J. Vos, User interface level testing with TESTAR; what about more sophisticated action specification and selection?, in: SATToSE, 2014, pp. 60–78.
[250] C.J.H. de Souza, B.G.B. Legey, N.V. de Oliveira, B.J.R. de Souza, ATF-an end-to-end testing framework: experience report, in: Proceedings of the 8th Brazilian Symposium on Systematic and Automated Software Testing, 2023, pp. 99–101.
[251] V. Garousi, E. Yildirim, Introducing automated GUI testing and observing its benefits: an industrial case study in the context of law-practice management software, in: 2018 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW), IEEE, 2018, pp. 138–145.
[252] M. Felderer, R. Ramler, A multiple case study on risk-based testing in industry, Int. J. Software Tools Technol. Trans. 16 (2014) 609–625.
[253] G. Denaro, L. Guglielmo, L. Mariani, O. Riganelli, GUI Testing in production: challenges and opportunities, in: Companion Proceedings of the 3rd International Conference on the Art, Science, and Engineering of Programming, 2019, pp. 1–3.
[254] A.M. Fard, A. Mesbah, E. Wohlstadter, Generating fixtures for javascript unit testing (t), in: 2015 30th IEEE/ACM International Conference on Automated Software Engineering (ASE), IEEE, 2015, pp. 190–200.
[255] M. Pradel, P. Schuh, G. Necula, S. Koushik, EventBreak: analyzing the responsiveness of user interfaces through performance-guided test generation, ACM SIGPLAN Notices 49 (10) (2014) 33–47.
[256] L. Mariani, M. Pezzè, O. Riganelli, M. Santoro, Link: exploiting the web of data to generate test inputs, in: Proceedings of the 2014 International Symposium on Software Testing and Analysis, 2014, pp. 373–384.
[257] M. Nolting, J.E. von Seggern, Context-based A/B test validation, in: Proceedings of the 25th International Conference Companion on World Wide Web, 2016, pp. 277–278.
[258] M.S. Raunak, D.R. Kuhn, R. Kacker, Combinatorial testing of full text search in web applications, in: 2017 IEEE International Conference on Software Quality, Reliability and Security Companion (QRS-C), IEEE, 2017, pp. 100–107.
[259] H.H. Deyab, R.B. Atan, Orchestration framework for automated ajax-based web application testing, in: 2015 9th Malaysian Software Engineering Conference (MySEC), IEEE, 2015, pp. 1–6.
[260] J. Polpong, S. Kansomkeat, Syntax-based test case generation for web application, in: 2015 International Conference on Computer, Communications, and Control Technology (I4CT), IEEE, 2015, pp. 389–393.
[261] R. Gupta, N. Bajpai, A keyword-driven tool for testing web applications (KeyDriver), IEEE Potentials 33 (5) (2014) 35–42.
[262] C. Sung, M. Kusano, N. Sinha, C. Wang, Static DOM event dependency analysis for testing web applications, in: Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering, 2016, pp. 447–459.

[263] M. Biagiola, A. Stocco, F. Ricca, P. Tonella, Dependency-aware web test generation, in: 2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST), IEEE, 2020, pp. 175–185.
[264] N. Li, A. Escalona, T. Kamal, Skyfire: model-based testing with cucumber, in: 2016 IEEE International Conference on Software Testing, Verification and Validation (ICST), IEEE, 2016, pp. 393–400.
[265] S. Hallé, N. Bergeron, F. Guerin, G. Le Breton, Testing web applications through layout constraints, in: 2015 IEEE 8th International Conference on Software Testing, Verification and Validation (ICST), IEEE, 2015, pp. 1–8.
[266] M. Muneer, U. Rasheed, M.M. Hussain, A meta-model to support compatibility testing of cross-browser web application, in: 2025 6th International Conference on Advancements in Computational Sciences (ICACS), IEEE, 2025, pp. 1–8.
[267] M. Fuksa, S. Speth, S. Becker, ViMoTest: a tool to specify ViewModel-based GUI test scenarios using projectional editing, in: 2025 IEEE Conference on Software Testing, Verification and Validation (ICST), IEEE, 2025, pp. 752–756.
[268] S. Di Meglio, L.L.L. Starace, S. Di Martino, E2E-loader: a tool to generate performance tests from end-to-end GUI-level tests, in: 2025 IEEE Conference on Software Testing, Verification and Validation (ICST), IEEE, 2025, pp. 747–751.
[269] M. Fuksa, S. Speth, S. Becker, Applicability of the ViMoTest approach for automated GUI testing: a field study, in: 2023 ACM/IEEE International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C), IEEE, 2023, pp. 821–830.
[270] E. Battista, S.D. Martino, S. Di Meglio, F. Scippacercola, L.L. Lucio Starace, E2E-loader: a framework to support performance testing of web applications, in: 2023 IEEE Conference on Software Testing, Verification and Validation (ICST), 2023, pp. 351–361. https://doi.org/10.1109/ICST57152.2023.00040
[271] J.N. Lo, S.-J. Lee, An empirical study on web GUI load testing and the hybrid with HTTP requests, in: 2023 IEEE International Conference on e-Business Engineering (ICEBE), IEEE, 2023, pp. 217–224.
[272] E. Pelivani, B. Cico, A comparative study of automation testing tools for web applications, in: 2021 10th Mediterranean Conference on Embedded Computing (MECO), IEEE, 2021, pp. 1–6.
[273] R.B. Bahaweres, E. Oktaviani, L.K. Wardhani, I. Hermadi, A. Suroso, I.P. Solihin, Y. Arkeman, Behavior-driven development (BDD) cucumber katalon for automation GUI testing case CURA and swag labs, in: 2020 International Conference on Informatics, Multimedia, Cyber and Information System (ICIMCIS), IEEE, 2020, pp. 87–92.
[274] D. Garg, A. Singhal, A. Bansal, A framework for testing web applications using action word based testing, in: 2015 1st International Conference on Next Generation Computing Technologies (NGCT), IEEE, 2015, pp. 593–598.
[275] E. Habibi, S.-H. Mirian-Hosseinabadi, Event-driven web application testing based on model-based mutation testing, Inf. Softw. Technol. 67 (2015) 159–179.
[276] A. Donvir, A comparative analysis of JavaScript unit and end-to-end testing frameworks: enhancing quality assurance in modern web applications, in: 2024 IEEE 16th International Conference on Computational Intelligence and Communication Networks (CICN), IEEE, 2024, pp. 1289–1296.
[277] C.-H. Liu, C.-J. Wu, H.-M. Chen, Testing of AJAX-based web applications using hierarchical state model, in: 2016 IEEE 13th International Conference on e-Business Engineering (ICEBE), IEEE, 2016, pp. 250–256.
[278] X.-F. Qi, Y.-L. Hua, P. Wang, Z.-Y. Wang, Leveraging keyword-guided exploration to build test models for web applications, Inf. Softw. Technol. 111 (2019) 110–119.
[279] X.-F. Qi, Z.-Y. Wang, J.-Q. Mao, P. Wang, Automated testing of web applications using combinatorial strategies, J. Comput. Sci. Technol. 32 (1) (2017) 199–210.
[280] B.K. Aichernig, R. Schumi, Property-based testing of web services by deriving properties from business-rule models, Software Syst. Model. 18 (2019) 889–911.
[281] W. Wang, S. Sampath, Y. Lei, R. Kacker, R. Kuhn, J. Lawrence, Using combinatorial testing to build navigation graphs for dynamic web applications, Software Testing Verif. Reliab. 26 (4) (2016) 318–346.
[282] O. Beroual, F. Guérin, S. Hallé, Searching for behavioural bugs with stateful test oracles in web crawlers, in: 2017 IEEE/ACM 10th International Workshop on Search-Based Software Testing (SBST), IEEE, 2017, pp. 7–13.
[283] M. Bures, Metrics for automated testability of web applications, in: Proceedings of the 16th International Conference on Computer Systems and Technologies, 2015, pp. 83–89.
[284] H.V. Nguyen, H.D. Phan, C. Kästner, T.N. Nguyen, Exploring output-based coverage for testing PHP web applications, Autom. Software Eng. 26 (2019) 59–85.
[285] Y. Zou, Z. Chen, Y. Zheng, X. Zhang, Z. Gao, Virtual DOM coverage for effective testing of dynamic web applications, in: Proceedings of the 2014 International Symposium on Software Testing and Analysis, 2014, pp. 60–70.
[286] D. Clerissi, G. Denaro, M. Mobilio, L. Mariani, Guess the state: exploiting determinism to improve GUI exploration efficiency, IEEE Trans. Software Eng. (2024), 50, 836–853.
[287] S. Hallé, G. Le Breton, F. Maronnaud, A.B. Massé, S. Gaboury, Exhaustive exploration of ajax web applications with selective jumping, in: 2014 IEEE Seventh International Conference on Software Testing, Verification and Validation Workshops, IEEE, 2014, pp. 243–252.
[288] K. Pilley, R. Mall, S. Biswas, V. Mamgain, R. Verma, S.K. Vishvakarma, A regression test case prioritization technique for web application using user session data, in: 2024 IEEE 4th International Conference on ICT in Business Industry & Government (ICTBIG), IEEE, 2024, pp. 1–8.
[289] R.K. Yandrapally, A. Mesbah, Fragment-based test generation for web apps, IEEE Trans. Software Eng. 49 (3) (2022) 1086–1101.
[290] F. Macklon, M. Viggiato, N. Romanova, C. Buzon, D. Paas, C.-P. Bezemer, A taxonomy of testable HTML5 canvas issues, IEEE Trans. Software Eng. 49 (6) (2023) 3647–3659.
[291] W. Ravelo-Méndez, C. Escobar-Velásquez, M. Linares-Vásquez, Kraken 2.0: a platform-agnostic and cross-device interaction testing tool, Sci. Comput. Program. 225 (2023) 102897.
[292] N. Yousaf, F. Azam, W.H. Butt, M.W. Anwar, M. Rashid, Automated model-based test case generation for web user interfaces (WUI) from interaction flow modeling language (IFML) models, IEEE Access 7 (2019) 67331–67354.
[293] F. Dias, A.C.R. Paiva, Pattern-based usability testing, in: 2017 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW), IEEE, 2017, pp. 366–371.